FPT UNIVERSITY

MINISTRY OF EDUCATION AND TRAINING
FPT UNIVERSITY

Capstone Project Document

AI-Based Abnormal Behavior Detection from Surveillance Cameras

Group Name	**GFA25AI06**
Group Members:	Nguyen Hoang Gia Vinh – Leader – SE170445 Bui Vinh Loc – Member – QE170176 Nguyen Hoang Khoi Minh – Member - SE161703
Supervisor:	Le Thanh Hai
Capstone Project code:	FA25AI47

Ho Chi Minh, November 2025

Acknowledgement

The completion of this capstone project would not have been possible without the invaluable support and guidance of many individuals and organizations.

First and foremost, we would like to express our deepest gratitude to our supervisor, **Mr. Lê Thanh Hải**, for his unwavering support, expert guidance, and constructive feedback throughout the development of our **AI-Based Abnormal Behavior Detection from Surveillance Cameras**. His mentorship was instrumental in helping us navigate the complexities of Deep Learning architectures and system integration, ensuring the project remained on the right track.

We would like to extend our sincere appreciation to the **review committee members**, whose insightful comments and constructive criticism during the review sessions have significantly improved the quality of our project. Their expertise and thorough evaluation have helped us refine our approach, particularly in optimizing model performance and deployment strategies.

We are particularly grateful to **FPT University** for providing us with the educational foundation and the necessary resources to undertake this challenging project. The university's commitment to fostering innovation and the practical application of artificial intelligence has been instrumental in shaping our approach to this research.

Special thanks go to the researchers and authors of the **UCF-Crime** and **XD-Violence** datasets, as well as the open-source community. Their willingness to share data and tools has been crucial for training and validating our models, allowing us to develop a practical and effective solution for public safety.

This project represents not just our technical achievement, but also the collective effort and support of all these individuals and institutions.

Definition and Acronyms

Acronym	Definition	Acronym	Definition
AI	Artificial Intelligence	**DL**	Deep Learning
CV	Computer Vision	**CCTV**	Closed-Circuit Television
VAD	Video Anomaly Detection	**TAL**	Temporal Action Localization
CNN	Convolutional Neural Network	**YOLO**	You Only Look Once (Object Detector)
3D CNN	3D Convolutional Neural Network	**ViT**	Vision Transformer
mAP	mean Average Precision	**tIoU**	temporal Intersection over Union
ROI	Region of Interest	**NMS**	Non-Maximum Suppression
FPS	Frames Per Second	**GPU**	Graphics Processing Unit
VRAM	Video Random Access Memory	**API**	Application Programming Interface
RTSP	Real Time Streaming Protocol	**AMP**	Automatic Mixed Precision
FN	False Negative	**FP**	False Positive
TP	True Positive	**TN**	True Negative
AdamW	Adaptive Moment Estimation with Weight Decay	**SGD**	Stochastic Gradient Descent
UI	User Interface	**IDE**	Integrated Development Environment

Table 1. List of Definitions and Acronyms

Contents

Definition and Acronyms 2
Contents 3
List of Tables 4
List of Figures 5
1. PROJECT INTRODUCTION 6
1.1 Project Information 7
1.2 Project Overview 8
1.2.1 Context 8
1.2.2 Core Problem 9
1.2.3 Proposed Solution 10
1.3 Project Objective 12
1.3.1 Primary Goal 12
1.3.2 Specific Objectives 12
1.4 Problem Statement 14
1.4.1 Spatio-Temporal Modeling Challenges 14
1.4.2 Computational Efficiency for Long-Form Video 15
1.4.3 System Integration and Usability 15
1.5 Significance of the Project 16
1.5.1 Practical Impact on Surveillance and Safety 16
1.5.2 Methodological Contribution 17
1.5.3 Efficiency and Resource Awareness 18
1.5.4 Educational and Reproducible Value 18
1.6 Project Scope & Limitations 18
1.6.1 Project Scope 19
1.6.2 Project Limitations 19
1.6.3 Out of Scope 20
2. PROJECT MANAGAMENT PLAN 21
2.1 Teamwork 22
2.1.1 Team Structure and Roles 22
2.1.2 Communication Plan 22
2.2. Project Management Approach 24
3. EXISTING SYSTEM / STATE OF THE ART 27

3.1 Overview of the Field 28
3.1.1 Action Recognition in Surveillance Video 28
3.1.2 CNN-based Video Models 28
3.1.3 Summary 29
3.2 Key Studies and Theories 29
3.2.2. MobileNetV3-Small 30
3.2.3 YOLOv11 31
3.3 Justification for the Project Architecture 32
3.4. Benchmark Comparison 33
4. METHODOLOGY 34
4.1 Research Questions and Objectives 35
4.1.1 Research Questions 35
4.1.2 Technical Objectives 36
a. YOLOv11 person and fire detection 37
b. Fire ROI extraction and verification 37
c. Person-based abnormality triggers 37
d. Temporal smoothing of fire predictions 38
4.1.4 Clip Generation and Buffering 38
a. Trigger-centred sliding windows 38
b. Overlap control and redundancy reduction 38
d. Supervised clip labelling 39
4.1.5 Spatio-Temporal Modeling Layer 39
a. Dual-pathway feature extraction 39
b. Clip-level classification head 40
c. Integration with the fire branch 40
4.1.6 Post-Processing and Visualization 40
a. Multi-scale temporal Transformer encoder 40
b. Segment prediction heads 40
c. Post-processing and JSON alert generation 41
4.2 Data Collection and Preprocessing 41
4.2.1. Anomaly Detection Dataset 42
a. Fire Verification Sub-Dataset (Frame-Level) 42
b. Human-Action Sub-Dataset (Clip-Level) 43
4.2.2 Data Preprocessing Methodology 43

a. Frame Extraction and Basic Normalization 43
b. Fire-Verification Branch (MobileNetV3-Small) 43
c. Human-Action Branch (SlowFast + Temporal Transformer) 44
d. Dataset Split 44
4.2.3 Dataset Statistics and Distribution 45
4.2.4 Data Augmentation and Sampling Strategy 46
a. Spatial and Photometric Augmentation for Person-Centred Clips 46
b. Dedicated Fire / Non-Fire Verification Branch with Targeted Augmentation 47
c. Class-Balanced Sampling for Rare Anomalies 48
4.3 Model Architecture 49
4.3.1 SlowFast (Spatio-Temporal Feature Extractor) 50
4.3.2 Environmental Hazard Pipeline (MobileNetV3) 51
4.4 Training and Deployment Strategy 52
5. EXPERIMENTAL SETUP 52
5.1 Implementation Details 53
5.1.1 Hardware and Software Environment 53
5.1.2 Dataset Splitting Protocol 53
5.1.3 Training Procedure for Human Action Branch (SlowFast Backbone) 54
5.1.4 Training Procedure for MobileNetV3 – Small 54
5.1.5 Inference Pipeline 55
5.2 Evaluation Metrics 55
5.2.1 Classification Metrics 55
5.2.2 Computational Efficiency Metrics 57
5.2.3 Diagnostic Tools: Confusion Matrix 58
6. RESULTS AND DISCUSSION 58
6.1 Branch 1: Human Behavior Analysis (YOLOv11 Pose + SlowFast) 59
6.1.1. Training and Validation Convergence 59
6.1.2. Classification Performance Metrics 59
6.1.3. Class-wise Evaluation (Confusion Matrix) 60
6.2 Brance 2: Fire & Safety Monitoring (YOLOv11 + CNN) 61
6.2.1. Fire Object Detection (YOLOv11) 61
6.2.2. Fire Classification Verification (CNN) 62
6.3. System-wide Experimental Conclusion 64
6.4 Comparative Analysis: Success Cases vs. Failure Modes 64

6.4.1 Success Cases (Good Cases) 64
6.4.1.1 Fire CNN: Robust Verification (The "Red Shirt" Scenario) 64
6.4.1.2. SlowFast: Distinguishing Falls from Bending 65
6.4.2 Failure Modes (Bad Cases) 66
6.4.2.1. Fire CNN: False Positive Bias in Baseline Models 66
6.4.2.2. SlowFast: The "Active Normal" Confusion 67
6.4.2.3. Pipeline Dependency: Occlusion-Induced Misses 68
7. CONCLUSION 69
7.1. Summary of Findings 70
7.2. Limitations and Future Work 70
8. REFERENCE 71

List of Tables

Table 2.1. Team Structure and Responsibilities 26
Table 2.2. Project Planning and Execution Strategy 29
Table 4.2 Data Collection and Preprocessing 48
Table 4.2.1. Dataset Distribution by Class and Split 48
Table 4.2.3. Overall Dataset Distribution and Imbalance Analysis 53
Table 4.2.4. Configuration of Temporal Sampling and Training Augmentations 56
Table 4.4. Training and Deployment Hyperparameters Configuration 60

List of Figures

Figure 1.2.3 System Architecture Overview 13
Figure 4.2.2. Sample Data with Bounding Box Placement for Pose Detection.....52
Figure 4.2.3. Frames and Sequences Analysis.....53
Figure 4.3.1. Dual-Pathway Mechanism of the SlowFast Network 58
Figure 4.3.2. Modified MobileNetV3-Small with Custom Classifier Head for Fire Detection.....60
Figure 6.1.1 Binary cross-entropy loss with 14 epochs.....67
Figure 6.1.2.a. Validation Accuracy Curve over Training Epochs.....68
Figure 6.1.2.b. Validation F1 Score Curve over Training Epochs.....68
Figure 6.1.3. Confusion Matrix of Anomaly Detection on Test Set 69
Figure 6.2.1. Stabilization of Box Regression and Classification Loss.....70
Figure 6.2.2.a Training and Validation Accuracy of the Fire CNN Classifier 71
Figure 6.2.2.b Training and Validation Loss of the Fire CNN Classifier 71
Figure 6.2.2.c. Confusion Matrix for Fire vs. Non-Fire Verification. 72
Figure 6.4.1.1. The specialized CNN successfully filters non-fire visual artifacts. 73
Figure 6.4.1.2 High sensitivity to 'Fall' and 'Fighting' events in the Pose-Augmented SlowFast model. 74
Figure 6.4.2.1. Inability to distinguish Background from Fire in the initial CNN 75
Figure 6.4.2.2. Misclassification of benign rapid movements as 'Fighting' due to kinetic similarity. 76
Figure 6.4.2.3 Occlusion-induced detection failure leading to a False Negative 77

1. PROJECT INTRODUCTION

1.1 Project Information

Project Title: AI-Based Abnormal Behavior Detection from Surveillance Cameras

This capstone project develops an AI-based surveillance system that automatically analyses CCTV videos, detects abnormal human behaviors and hazardous events, and highlights suspicious temporal segments for human review. Instead of relying solely on operators who must visually inspect dozens of live streams, the system leverages deep learning to provide near real-time alerts and a time-aligned incident timeline, allowing security staff to focus on verification and response rather than repetitive monitoring [1], [2].

The proposed system follows a three-stage hybrid deep-learning architecture:

- Stage 1 – YOLOv11: Frame-level abnormality trigger
  A YOLOv11 detector scans each frame to detect people and fire-like regions. The YOLO family has become the de facto standard for real-time object detection thanks to its single-stage design and excellent speed–accuracy trade-off [5], [6]. Recent studies show that next-generation variants such as YOLOv10 and YOLOv11 further improve accuracy and latency on high-resolution streams, making them attractive for surveillance applications [7], [8]. In this project, YOLOv11 is used to:
  - Flag frames that potentially contain abnormal events (e.g., a person falling, multiple people in close interaction, fire).
  - Produce person-centred and fire-centered crops that are later processed by the action-recognition and fire-verification modules.
- Stage 2A – SlowFast: Spatio-temporal action recognition
  Person-centered clips extracted around suspicious frames are fed into a SlowFast network. SlowFast uses a dual-pathway design: a low-framerate “Slow” pathway for semantic information and a high-framerate “Fast” pathway for motion details [4]. This architecture has proven highly effective for video action recognition and is therefore adopted as the main spatio-temporal backbone for classifying clips into *Fighting*, *Fall*, and *Normal* activities.
- Stage 2B – MobileNetV3-Small: Fire verification
  YOLOv11 may produce false positives when objects share similar colors or shapes with fire (e.g., bright orange clothing, traffic lights, reflections). To reduce such errors, a lightweight MobileNetV3-Small classifier performs binary fire vs. non-fire classification on YOLO-cropped regions [9]. Vision-based fire detection surveys highlight the effectiveness of deep CNNs in distinguishing actual flames from visually similar but harmless patterns [11], [12]. This verification stage prevents the system from raising unnecessary fire alarms.
- Output – Temporal aggregation, alert generation, and user interface
  The last stage transforms clip-level predictions into temporally precise incident segments and exposes them to a Flask-based web dashboard. Sequences of predictions from SlowFast and the fire branch are fed into a lightweight temporal transformer encoder inspired by recent Transformer-based approaches for temporal action proposal generation

and localization. The encoder smooths scores over time, merges consecutive high-confidence predictions, and outputs action segments annotated with:
    - Event label (*Fire*, *Fighting*, *Fall*),
    - Start and end timestamps, and
    - Confidence scores.

These segments are serialized as JSON alerts and served through an API to the frontend, which displays a color-coded timeline alongside the video. Users can jump directly to interesting moments instead of watching the entire recording.

The current prototype focuses on four classes:

- Fire – visible flames or dense fire patterns indicating a fire hazard.
- Fighting – aggressive physical interactions such as punching, kicking, or wrestling.
- Fall – sudden loss of balance followed by a person collapsing onto the ground.
- Normal – everyday activities that do not require intervention.

These classes are widely studied in video anomaly detection and are highly relevant in environments such as campuses, hospitals, and public facilities [1], [3], [17]. By concentrating on this focused set of behaviors, the project keeps dataset and model complexity manageable while still addressing high-impact safety scenarios.

1.2 Project Overview

The goal of this section is to situate the project within the broader landscape of intelligent video surveillance and to describe, at a high level, why the system is needed and how it is conceptually designed.

1.2.1 Context

Modern surveillance networks continuously capture vast quantities of video from indoor and outdoor cameras. Human operators are expected to monitor multiple screens, recognize suspicious situations, and respond promptly. In practice, this manual process is:

- Labor-intensive – operators must monitor many streams in parallel.
- Error-prone – fatigue and attention lapses can cause missed incidents.
- Difficult to scale – adding more cameras linearly increases monitoring load.

Recent reviews on video anomaly detection emphasize that deep learning has become the dominant paradigm for automatically identifying unusual behaviors or events in surveillance footage [1], [2], [3]. These methods learn normal patterns from large volumes of video and flag deviations such as fights, falls, theft, or hazardous events. Compared with hand-crafted features or simple motion detectors, modern deep models better capture complex spatial and temporal patterns in crowded or cluttered scenes [1], [3].

Beyond general anomaly detection, several domain-specific hazards have been extensively investigated:

- Fire detection in video – Deep CNN and YOLO-based methods can detect fires and flames earlier and more robustly than traditional sensor-only approaches, especially in large open spaces [11], [12], [13], [14].
- Fall detection – Vision-based fall-detection systems use deep learning to distinguish falls from other everyday movements, supporting safety in hospitals, elderly-care centers, and workplaces [10], [15], [16].
- Violence and fighting detection – Recent surveys on video violence detection review models for detecting aggressive interactions in CCTV footage, combining spatial appearance and motion cues [17], [18].

The proposed project builds on insights from these research threads and adapts them into a single, unified system focused on detecting *Fire*, *Fighting*, and *Fall* events in realistic campus-like surveillance scenarios.

1.2.2 Core Problem

1. **Stage 1 – YOLOv11-based video processing and ROI generation**
    a. Each input video is decoded into frames and processed by YOLOv11.
    b. The detector localizes people and fire-like regions with high speed and accuracy, leveraging advances in the YOLO family up to YOLOv11 [6], [7].
    c. Potentially abnormal frames (e.g., frames containing people with unusual posture or fire-like regions) trigger the creation of temporal clips and image crops:
        i. Person-centered clips for SlowFast (Fighting/Fall/Normal).
        ii. Fire-centred crops for MobileNetV3-Small (Fire/Non-Fire).
2. **Stage 2 – Deep action and fire understanding**
    a. **Stage 2A – SlowFast:**
    A SlowFast network processes each person-centered clip and outputs a class label (*Fighting*, *Fall*, *Normal*) together with confidence scores. The dual-pathway design allows the network to capture both semantic context and fast motion patterns [4].
    b. **Stage 2B – MobileNetV3-Small:**
    MobileNetV3-Small receives fire-centered crops from YOLOv11 and determines whether they truly correspond to fire or to visually similar non-fire objects [9], [11]. Only regions classified as fire proceed as confirmed fire events.
3. **Output – Temporal aggregation, JSON alerts, API, and Timeline UI**
    a. For each video, predictions from Stage 2 form time-ordered sequences of clip-level scores (for Fighting/Fall/Normal) and frame-level fire events.
    b. The encoder smooths noisy predictions, merges adjacent segments with consistent labels, and refines their start and end boundaries.
    c. The output is a set of action segments, each defined by:
        i. label ∈ {Fire, Fighting, Fall},
        ii. start_time, end_time,

iii. confidence.

d. Each segment is serialized into a JSON alert and stored or transmitted via a Flask REST API to the frontend.
e. The web dashboard visualizes alerts as a color-coded timeline synchronized with video playback. Users can click on any segment to jump directly to the corresponding event.

This architecture directly addresses the gaps identified earlier: YOLOv11 focuses computation on relevant regions, SlowFast and MobileNetV3-Small provide robust classification, time-stamped incident summaries [1], [4].

1.2.3 Proposed Solution

The system architecture is designed as a sequential, multi-stage pipeline that processes raw surveillance footage into actionable real-time alerts. The data flow through the five main stages as illustrated in **Figure 1.2.3**:

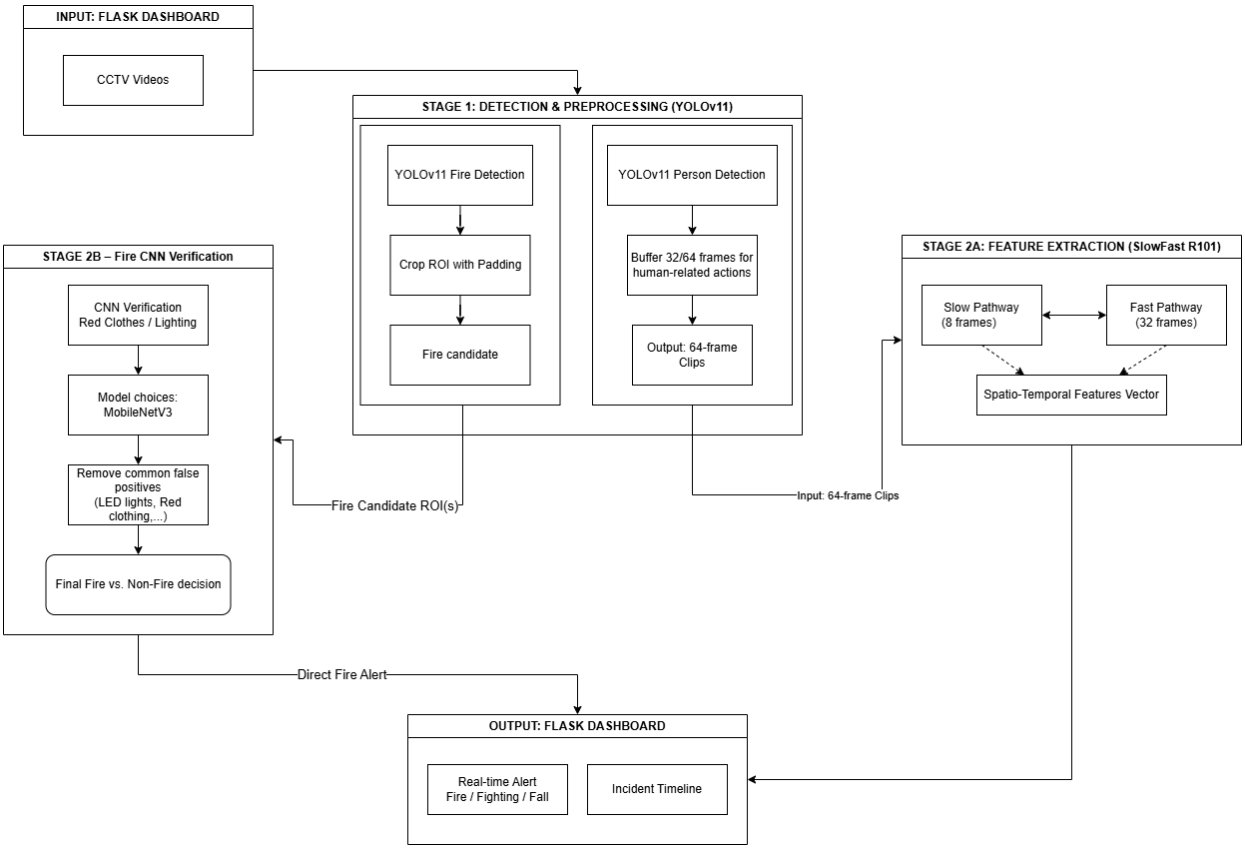

Figure 1.2.3. System Architecture Overview

Stage 1: YOLO11 – Abnormality Trigger (Frame-Level)

YOLO11 scans each frame to detect potential abnormal visual cues such as:

- Fire regions,Violent/aggressive poses,
- Falling posture or collapsed person,
- Sudden body movement patterns.

When YOLO11 detects abnormalities:

- A short clip (e.g., 16–32–64 frames) around the abnormal frame is automatically extracted.
- This clip is forwarded to the SlowFast model.

Using YOLO11 in this way avoids analyzing the full video with 3D CNNs, saving computation time and focusing the pipeline on truly relevant segments, which is consistent with recommendations for efficient anomaly-detection systems [2], [4], [7].

Stage 2A: SlowFast – Action Classification

SlowFast receives clips extracted by YOLO11 and predicts one of the three target actions:

- Fighting
- Fall
- Normal

This stage answers the question:

“What action is happening in the extracted clip?”

The dual-pathway architecture of SlowFast enables the model to capture both spatial semantics and fine-grained motion, which has been shown to be highly effective for video action recognition [3].

Stage 2B: MobileNet – Fire Classification

To reduce false alarms caused by objects that share similar color or brightness characteristics with fire (such as red clothing, orange lights, or reflections), YOLO11 detections are passed to a secondary verification module implemented using MobileNetV3-Small. This lightweight CNN is pretrained on ImageNet (IMAGENET1K_V1) and is specifically designed for efficiency in real-time vision pipelines. MobileNetV3 combines depth wise-separable convolutions with squeeze-and-excitation blocks to achieve high accuracy while maintaining a compact computational footprint—making it well suited for CCTV-based fire detection systems. This stage answers the question:

“Is the detected region truly fire?”

Only detections confirmed by MobileNetV3 are forwarded as valid fire events.

End-to-End Functionality

The complete system operates as follows:

1. User uploads a video via the Flask dashboard.

2. YOLO11 detects abnormal frames.
3. Relevant temporal clips are extracted around those frames.
4. SlowFast and MobileNet classifies actions and extracts temporal features.
5. Results (timeline + action labels + confidence) are visualized on the dashboard.

This end-to-end design reduces manual monitoring effort and provides reliable, automated incident detection, in line with current research directions for intelligent surveillance [4], [5], [7].

1.3 Project Objective

This section formalizes what the project aims to achieve.

1.3.1 Primary Goal

The primary goal of this project is to design, implement, and deploy a practical AI-based abnormal behavior detection system that:

- Automatically detects *Fire*, *Fighting*, and *Fall* events in untrimmed CCTV videos.
- Produces precise temporal localization (start and end times) for each detected event using a temporal Transformer-based aggregation module.
- Achieves near real-time performance on a GPU workstation equipped with an NVIDIA RTX 3060.
- Presents results through an intuitive Flask-based web dashboard that offers both incident timelines and visual overlays.

By achieving this goal, the system aims to support more proactive incident detection and faster response in real surveillance operations, complementing existing human-based workflows [2], [3], [17].

1.3.2 Specific Objectives

To realize the primary goal, the project is decomposed into the following specific technical objectives:

1. **Design a hybrid architecture for abnormal-event understanding**
    a. Combine YOLOv11, SlowFast, and MobileNetV3-Small into a cohesive end-to-end pipeline for abnormal behavior detection in untrimmed surveillance videos.
    b. Clearly separate responsibilities: YOLOv11 serves as a frame-level abnormality trigger and region-of-interest (ROI) generator, SlowFast provides spatio-temporal action recognition for *Fighting*, *Fall*, and *Normal*, while MobileNetV3-Small performs appearance-based fire verification [4], [6], [9].
2. **Develop an efficient clip and ROI generation strategy based on YOLOv11 triggers**
    a. Use YOLOv11 detections to select informative temporal windows around potentially abnormal frames instead of naïvely sliding fixed windows over the entire video.

   b. Tune window length, stride, and minimum overlap so that the system reliably covers short, rare events while avoiding an excessive number of redundant clips sent to SlowFast [1], [6].
3. **Fine-tune SlowFast for abnormal action classification**
   a. Start from a pre-trained SlowFast model originally trained on large-scale action-recognition datasets and adapt it to the custom abnormal-behavior dataset focusing on *Fire*-context clips, *Fighting*, *Fall*, and *Normal* [3], [4].
   b. Explore training strategies such as partial versus full fine-tuning, class balancing, and data augmentation (spatial transforms, temporal jittering, and color perturbations) to improve robustness to viewpoint changes, occlusion, and motion variability [3], [17].
4. **Build a robust fire-verification module with MobileNetV3-Small**
   a. Implement a lightweight appearance-based verification stage using MobileNetV3-Small to filter out false-positive fire detections produced by YOLOv11 [9].
   b. Train this module to perform binary fire versus non-fire classification on YOLO-extracted regions, suppressing colour-based confounders such as red clothing, lamps, and reflections, guided by recent deep-learning fire-detection surveys [11], [12].
5. **Implement a temporal Transformer head for event segmentation and alert generation**
   a. Design a lightweight temporal Transformer encoder that aggregates clip-level predictions from SlowFast and fire-verification outputs into coherent action segments with explicit start and end timestamps.
   b. Apply temporal smoothing and simple boundary-refinement rules to merge fragmented detections and output a structured list of segments (*Fire*, *Fighting*, *Fall*) that can be serialized as JSON alerts and served through a REST API to the frontend timeline UI.
6. **Define a comprehensive evaluation protocol**
   a. Measure classification quality using metrics such as Accuracy, Precision, Recall, and F1-score for each target class, with particular attention to rare but critical events such as fighting and falls [1], [2].
   b. Evaluate temporal localization quality using segment-level mean Average Precision (mAP) on a held-out test set, and analyse common failure modes—for example, confusion between *Fall* and *Normal*, or misclassification of non-violent interactions as *Fighting*—to guide further model refinement [2], [15].
7. **Deploy the complete system as a Flask dashboard with GPU acceleration**
   a. Package the trained models and preprocessing pipeline into a Flask-based web application capable of running efficiently on an NVIDIA RTX 3060 (12 GB VRAM), targeting near real-time processing for typical surveillance resolutions [2], [3].
   b. Provide an operator-oriented interface that supports video upload, progress indication, and interactive visualization of detected events through bounding boxes and a colour-coded timeline synchronized with playback, facilitating practical use in security-monitoring scenarios [3], [17].

Collectively, these objectives ensure that the project delivers not just a theoretical model but a fully implemented, evaluated, and deployable abnormal-behavior detection system.

1.4 Problem Statement

Although deep learning has significantly improved video understanding, building a reliable abnormal-behavior detection system for real-world CCTV footage is still challenging. The project must simultaneously:

- Model complex spatio-temporal behaviors such as Fighting and Fall, which are defined by motion patterns over time rather than single frames [1], [4].
- Maintain computational efficiency on long untrimmed videos that mostly contain normal activities [1], [3].
- Integrate multiple models—YOLOv11, SlowFast, MobileNetV3-Small, and a temporal Transformer head—into a coherent, user-friendly pipeline that outputs time-stamped JSON alerts and a timeline-based dashboard [2], [11].

The following subsections detail the key challenges that motivate the proposed architecture.

1.4.1 Spatio-Temporal Modeling Challenges

Abnormal behaviors such as Fighting and Fall are inherently spatio-temporal patterns. They are recognized not only from the visual appearance of individual frames but from how body posture and interactions evolve over time:

- A fall involves a characteristic loss of balance followed by a rapid downward motion and a period of lying on the ground.
- A fight typically exhibits repeated aggressive gestures, sudden accelerations, and close-range interactions between multiple individuals.

Surveys on video anomaly detection emphasize that many anomalies can only be distinguished when temporal context is considered, especially in crowded or cluttered scenes [1], [2]. Traditional 2D CNNs extended to video (e.g., stacking frames and applying 2D convolutions) often struggle to capture long-range temporal structures efficiently [1], [3]. More recent 3D CNNs and two-stream architectures significantly improve spatio-temporal modeling but can be computationally demanding for continuous surveillance [3], [4].

SlowFast directly addresses this challenge by using a dual-pathway design (slow semantic pathway + fast motion pathway), achieving state-of-the-art results on video benchmarks with a favorable accuracy–efficiency trade-off [4]. In this project, SlowFast is fine-tuned on person-centred clips to classify *Fighting*, *Fall*, and *Normal* behaviors. However, SlowFast alone is not sufficient:

- Fire anomalies are better characterized by appearance than by body motion, so the system introduces a MobileNetV3-Small fire-verification branch that focuses on texture and colour patterns to distinguish real fire from visually similar but harmless objects [9], [11], [12].
- Per-clip predictions from SlowFast and the fire branch must be aggregated over time to produce coherent incident segments with clear start and end boundaries. This motivates a temporal Transformer head inspired by recent Transformer-based temporal action proposal and localization frameworks.

Effectively adapting these models to the specific abnormal behaviors targeted in this project—Fire, Fighting, and Fall—while keeping them interpretable and efficient remains a non-trivial research problem.

1.4.2 Computational Efficiency for Long-Form Video

Continuous CCTV monitoring produces large volumes of video, typically dominated by normal behavior or even empty scenes. Directly applying heavy 3D CNNs or Transformers to every frame, or to dense sliding windows, is computationally infeasible for near real-time operation, even on modern GPUs [1], [4]. In practice, the system must strike a balance between:

- Coverage – ensuring that short, rare anomalies (e.g., a sudden fall or a small fire starting in the corner of the frame) are not missed.
- Cost – avoid unnecessary computation on uninformative regions and redundant clips.

The proposed architecture addresses this by using YOLOv11 as a front-end filter:

- YOLOv11 scans each frame to detect people and fire-like regions with high speed and accuracy, benefiting from improvements in the YOLO family up to YOLOv10/11 [6], [7].
- Only frames containing relevant objects or configurations (e.g., people with unusual posture, potential fire regions) trigger clip generation for SlowFast or fire verification.

This design reduces the number of clips processed by the SlowFast network and focuses computation on frames where abnormal behavior is most likely to occur [3], [6].

For deployment, the system is optimized to run on an NVIDIA RTX 3060 (12 GB VRAM), targeting near real-time or faster-than-real-time processing for typical surveillance resolutions. This involves careful tuning of batch sizes, clip lengths, mixed precision, and I/O pipelines to prevent bottlenecks between CPU-based video decoding and GPU-based inference [4], [6].

1.4.3 System Integration and Usability

Even state-of-the-art models can have limited practical value if they are not integrated into a coherent system that operators can easily use. In real monitoring environments, security staff need more than just raw predictions: they require an end-to-end workflow that transforms raw video into actionable insights.

The proposed system must integrate:

- Video I/O and preprocessing – decoding video files, extracting frames, and handling different resolutions and frame rates.
- Stage 1 (YOLOv11) – frame-level detection to generate person ROIs and fire ROIs [5], [6], [8].
- Stage 2A (SlowFast) – spatio-temporal action recognition on person-centred clips, outputting class labels and confidence scores [4].
- Stage 2B (MobileNetV3-Small) – fire vs. non-fire verification on YOLO crops to reduce false alarms [9], [11].
- Backend services – logging alerts, storing metadata, and exposing them through a Flask REST API.
- Output - Frontend dashboard – providing video upload, progress indication, and a synchronized timeline UI for visual review.

From a usability perspective, operators should be able to:

- Upload a video or select a stream,
- Monitor processing status,
- See a color-coded timeline showing where *Fire*, *Fighting*, and *Fall* events occur,
- Click on timeline segments to jump directly to the corresponding moments, and
- Review bounding boxes and labels overlaid on the video.

Literature on intelligent video surveillance emphasizes that such interpretable interfaces and clear incident summaries are essential for practical adoption, as they bridge the gap between complex models and human decision-making [2], [3], [17]. The project therefore considers system integration and usability as core parts of the problem statement, not just optional engineering details.

1.5 Significance of the Project

The significance of the proposed abnormal-behavior detection system can be analyzed along four dimensions: **practical impact**, **methodological contribution**, **efficiency**, and **educational value**.

1.5.1 Practical Impact on Surveillance and Safety

The proposed system targets three high-impact safety scenarios in CCTV environments: **Fire**, **Fighting**, and **Fall**, with *Normal* as the background class. Automatically detecting and localising these events brings several practical benefits:

Earlier detection of hazardous situations

- Vision-based fire detection complements or even outperforms traditional sensor-only approaches, especially in large open spaces where smoke or temperature sensors may react late [11], [12].

- Accurate fall detection supports safety in hospitals, elderly-care homes, and workspaces, where timely assistance can significantly reduce injury severity [10], [15].

Support for crime prevention and public security

- Fighting and violent behaviors often precede more serious incidents. Automatic detection enables security staff to respond faster and allocate resources more effectively [17], [18].

Reduced operator workload

- Instead of passively watching long video streams, operators can focus on time-stamped alerts and color-coded timelines that highlight abnormal segments (Fire/Fighting/Fall).
- This aligns with trends in video anomaly detection that emphasise actionable, interpretable outputs rather than only global video-level anomaly scores [1], [2], [3].

Overall, the system aims to turn raw CCTV footage into **actionable intelligence**, helping organizations improve situational awareness and response time with minimal changes to existing camera infrastructure.

1.5.2 Methodological Contribution

From a methodological viewpoint, the project illustrates how to compose a three-stage hybrid architecture tailored to abnormal behavior detection in untrimmed surveillance videos:

Stage 1 – YOLOv11 front-end detection and ROI generation

- Uses a modern YOLO variant as a real-time detector for people and fire-like regions, leveraging advances from earlier YOLO generations [5], [6], [7].
- Acts as a filter that reduces the search space for heavier models by focusing on frames and regions that are likely to contain abnormal activity [3], [6].

Stage 2 – Dual-branch feature extraction and classification

- **SlowFast** processes person-centered clips to classify *Fighting*, *Fall*, and *Normal* behaviors, benefiting from its dual-pathway (slow + fast) design for capturing semantics and motion [4].
- **MobileNetV3-Small** serves as a lightweight fire-verification classifier operating on YOLO crops to distinguish real fire from visually similar but harmless objects [9], [11].

Output – User Interface and alert generation

- The segments are exported as JSON alerts and exposed via a Flask REST API, enabling downstream visualization and integration with external systems.

While each model family (YOLO, SlowFast, MobileNet, Transformers) has been explored extensively in prior research, combining them into a practical, end-to-end abnormal-behavior detection pipeline for CCTV footage is a non-trivial integration effort and constitutes a key contribution of this project [1], [3], [13].

1.5.3 Efficiency and Resource Awareness

A central design principle of the system is **efficient use of computational resources**:

- The architecture avoids running heavy 3D CNNs or Transformers on every frame by using YOLOv11 as a gatekeeper, passing only promising segments to SlowFast and the temporal head [3], [6].
- MobileNetV3-Small is chosen as the fire-verification backbone because it is highly efficient on edge/class-level tasks while maintaining adequate accuracy [9], [10].

This multi-stage structure is consistent with recent works on efficient video anomaly detection, which recommend allocating minimal computation to broad screening and focusing heavier models on candidate regions [1], [3]. In practical terms, it makes the system more realistic to deploy on a single NVIDIA RTX 3060 (12 GB) while still achieving near real-time performance on typical CCTV resolutions.

1.5.4 Educational and Reproducible Value

The project also offers educational and reusable value:

- It covers the full lifecycle of an AI system: literature review, problem formulation, dataset construction, model selection, training, evaluation, and deployment as a web application.
- The codebase is modular: each stage (YOLOv11, SlowFast, MobileNetV3-Small, temporal Transformer, Flask dashboard) is implemented as a separate component with clear interfaces, making it easy to replace or extend individual modules.
- Future students or practitioners can reuse the pipeline to explore:
    - Alternative action-recognition backbones (e.g., Transformer-based video encoders).
    - Additional anomaly classes such as theft, loitering, or road-traffic accidents.
    - Different deployment environments (edge devices, cloud servers).

Such modular and well-documented systems are valuable teaching tools and starting points for applied research on intelligent video surveillance [2], [3], [17].

1.6 Project Scope & Limitations

This section clearly defines the boundaries of our AI-Based Abnormal Behavior Detection System, outlining both the specific areas of focus and the acknowledged limitations of the current implementation. Understanding these parameters is crucial for managing expectations regarding system performance and ensuring effective project delivery.

1.6.1 Project Scope

This project focuses on vision-based abnormal behavior detection in surveillance videos captured from fixed-position cameras. The key elements of the scope are:

- **Target behaviours**

    - *Fire* – visible flames or strong fire patterns in the scene.
    - *Fighting* – aggressive physical interactions involving at least one person.
    - *Fall* – sudden loss of balance leading to a person collapsing to the ground.
    - *Normal* – everyday non-hazardous activities, treated as the background class.
- **Input data**
    - Untrimmed CCTV-like videos with typical indoor or semi-outdoor scenes.
    - Single-camera view at a time; multi-camera synchronization is not considered.
- **Model components**
    - YOLOv11 for frame-level detection and ROI generation.
    - SlowFast for person-centered action recognition.
    - MobileNetV3-Small for fire vs. non-fire verification on YOLO crops.
    - A temporal Transformer head for converting clip-level predictions into labelled action segments with start and end timestamps [4], [6], [9].
- **Deployment environment**
    - Single-GPU workstation (NVIDIA RTX 3060, 12 GB VRAM) running a Flask backend and a web-based frontend for video upload and result visualization.
- **Outputs**
    - Structured JSON alerts containing {label, start_time, end_time, confidence} for each detected event.
    - A color-coded timeline UI synchronized with video playback, allowing operators to jump directly to abnormal segments.

Within these boundaries, the project aims to deliver a working prototype that demonstrates the feasibility and effectiveness of the proposed three-stage pipeline.

1.6.2 Project Limitations

Despite its contributions, the current system has several limitations:

Sensitivity to occlusion and crowding

- In crowded scenes or with strong occlusion, YOLOv11 may miss people or fire regions, and SlowFast may not observe enough body motion to reliably distinguish falls or fights [1], [3].

Dependence on video quality and environment

- Performance can degrade under poor illumination, extreme viewpoints, low resolution, strong compression, or adverse weather. The system is tuned mainly for indoor or semi-outdoor campus-like settings and may not generalize perfectly to other domains without further fine-tuning [2], [11], [14].

Behavioral ambiguity

- Some movements are inherently ambiguous: fast sitting or kneeling may resemble a fall; playful pushing may look similar to violence. Even advanced deep models may misclassify such cases, causing false positives or false negatives [15], [18].

Scalability and throughput

- While the system is optimized for a single GPU, scaling to many high-resolution camera streams would require additional engineering, such as model compression, multi-GPU or distributed inference, and more sophisticated scheduling [3], [6].

Limited class set

- The current prototype models only *Fire*, *Fighting*, *Fall*, and *Normal*. Other relevant anomalies such as theft, loitering, vandalism, or traffic accidents are not explicitly recognized, which may limit applicability in some domains [2], [17].

Recognizing these limitations provides a realistic understanding of what the current system can achieve and highlights promising directions for future enhancement.

1.6.3 Out of Scope

The following aspects are intentionally excluded from this project:

Identity recognition and biometric analytics

- The system focuses on detecting *what* happens in the scene, not *who* is involved. Face recognition, person re-identification, age/gender estimation, and other biometric analytics are avoided due to privacy, ethical, and legal considerations [2], [17].

Audio-based analysis

- Only visual data from CCTV cameras are used. Audio cues such as screams, alarms, or glass breaking are not processed, even though they could provide useful complementary evidence [1], [18].

Direct integration with physical safety systems

- The prototype outputs software-level alerts and visualizations only. Integration with physical alarms (sirens, sprinklers), access-control systems, or automatic camera control (pan–tilt–zoom) is left for future work.

Comprehensive multi-camera management

- Features such as camera registration, user management, long-term storage policies, and cross-camera tracking are outside the scope. The system analyses one video (or one stream) at a time through the dashboard [3].

Broader anomaly categories

- The project does not attempt to cover all possible anomalies. Events like loitering, crowd panic, theft, or road-traffic violations would require additional datasets, labels, and model adaptations [2], [15].

These exclusions help focus the project on a manageable and well-defined problem while leaving clear avenues for future expansion.

2. PROJECT MANAGAMENT PLAN

2.1 Teamwork

2.1.1 Team Structure and Roles

Member	Student ID	Main Role	Responsibilities
Nguyen Hoang Gia Vinh	SE170445	- Team Leader - Project Manager - Coordinator	- Overseeing the project timeline, deliverables, and coordinating team communication. - Designing the end-to-end pipeline integration (YOLOv11 detection and ROI generation → MobileNetV3-Small fire verification → SlowFast R101 clips → Temporal Transformer head). - Managing the Flask backend logic to handle video ingestion and result aggregation. - Leading the writing of the final report and technical documentation.
Bui Vinh Loc	QE170176	- Machine Learning Model Developer - Full-stack Support	-Responsible for training and fine-tuning the SlowFast backbone for action recognition. - Tuning hyperparameters (learning rate, batch size) and handling GPU memory constraints (Gradient Accumulation, AMP). - Conducting quantitative experiments and analyzing metrics (mAP, F1-score). - Developing the Flask Dashboard UI for video upload and result visualization.
Nguyen Hoang Khoi Minh	SE161703	- Model Researcher - Data Collector - Full-stack Support	- Collecting and preprocessing surveillance datasets (e.g., aggregating clips, frame extraction, and normalization). - Implementing the **YOLO11** module for frame-level anomaly screening and clip generation. - Investigating state-of-the-art methods and benchmarking baselines.

Table 2.1. Team Structure and Responsibilities

2.1.2 Communication Plan

Effective communication is essential for coordinating work across data preparation, model training, and system integration. The team adopted the following communication strategy:

Weekly supervisor meetings

- **Frequency:** 2 times per week (typically Monday and Saturday evenings).
- **Format:** Online or on-campus meetings using Google Meet.
- **Content:** Progress updates for each work package (data, models, backend, frontend), discussion of encountered issues (e.g., GPU memory limits, convergence problems), and agreement on tasks for the next week.
- **Outcome:** Action items are summarized and recorded in a shared Google Doc right after each meeting so that all members have a clear to-do list.

Internal stand-up meetings

- **Frequency:** Short 15–20 minute stand-ups 3–4 times per week via Messenger/Discord.
- **Purpose:** Synchronize on day-to-day tasks such as: “Which experiment is YOLOv11 running today?”, “Which branch contains the latest Temporal Transformer code?”, or “Which videos still need ground-truth labels?”.
- **Benefit:** Early detection of blockers (e.g., bugs in dataloaders, training crashes) and quick re-assignment of tasks if one member is overloaded.

Collaboration tools and repositories

- **Source control:** GitHub is used as the central repository. Each major feature (YOLOv11 fire module, SlowFast training script, MobileNetV3 fire verifier, Stage-3 Transformer head, Flask UI) is developed in a dedicated branch and then merged via pull requests after code review.
- **Task tracking:** A shared Google Sheet / Docs is used as a lightweight Kanban board to track tasks, deadlines, and ownership. Tasks are grouped by work package: Data, Models, Backend, Frontend, Evaluation, and Documentation.
- **Documentation:**
    - Experimental logs (hyper-parameters, metrics, confusion matrices) are stored in structured markdown/CSV files inside a experiments/ folder.
    - Architecture diagrams, including the Stage-1 fire/ROI pipeline and the overall three-stage pipeline, are stored in a shared design folder and referenced directly in the report.
- **File sharing:** Large assets (raw videos, extracted frames, trained model checkpoints) are synchronized via Google Drive and carefully named (stage1_yolo11_fire_best.pt, slowfast_r101_abnormal_best.pth, etc.) to avoid confusion.

Decision-making process

- For technical decisions (e.g., clip length for SlowFast, number of layers in the Temporal Transformer head, or thresholds for alert generation), the member in charge prepares 1–2 options backed by small experiments. The team then decides collectively during the next stand-up.
- For scope or schedule changes (e.g., adding/removing features, adjusting milestones), the proposed change is first discussed inside the team, then validated with the supervisor during the weekly meeting to ensure alignment with course requirements.

This communication structure provides enough formality to keep the project on track while remaining lightweight and flexible for day-to-day development.

2.2. Project Management Approach

Function Information					Duration (Weeks)
No	**Component**	**Subcomponent**	**Function Information**	**To Do List**	
1	General	Research and planning	Define system architecture	- Define project goals: Define project goals: Build an AI-based abnormal behavior detection system. - Determine the data pipeline: Video → Frame Extraction → YOLO Trigger → SlowFast and CNN - Research suitable AI models (YOLO11, SlowFast). - Select main libraries (PyTorch, Ultralytics, PyTorchVideo, Flask).	01/09/2025 – 12/09/2025
2	Data Processing	Data Collection & Preprocessing	Prepare training data	- Collect datasets (UCF-Crime, XD-Violence, YouTube custom clips). - Preprocess videos: Frame extraction (30 FPS), Resizing (224x 224). - Implement YOLO11 inference to crop persons/ROI and filter empty frames. - Generate 64-frame clips for SlowFast input.	12/09/2025 – 13/10/2025
3	Model Development	Spatio-Temporal Modeling	Build feature extraction & classification model	- Fine-tune **SlowFast** backbone on custom classes (Fire, Fighting, Fall, Normal).	19/09/2025 – 03/10/2025

				- Validate feature extraction quality using Accuracy/F1-score. - Export feature banks for temporal modeling.	
		Temporal Localization Model	Build boundary regression model	- Optimize hyperparameters (learning rate, window size) for detection.	
4	System Development	Backend (FastAPI)	Develop API for AI pipeline	- Build a Flask API to handle video upload and frame buffering. - Integrate the full pipeline: YOLO11 trigger → SlowFast and CNN - Implement result aggregation logic (NMS, confidence thresholding).	12/10/2025 – 19/10/2025
		Frontend (Web Dashboard)	User interface development	- Design a Web Dashboard for users to upload surveillance videos. - Visualize detection results: Timeline, Bounding Boxes, and Action Labels.	
5	Testing & Evaluation	Model Testing	Validate model performance	- Run evaluation on the held-out Test Set. - Calculate metrics: mAP@tIoU (for localization) and Confusion Matrix (for classification).	19/10/2025 – 02/11/2025
		System Testing	Evaluate efficiency	- Measure inference speed (FPS) on RTX 3060. - Stress test with long untrimmed videos to ensure stability.	

6	Deployment	Web Deployment	Deploy the system	- Package the application for local deployment on GPU workstation. - Write installation scripts (requirements.txt, setup.sh).	02/11/2025 – 09/11/2025
7	Finalization & Reporting	Project Report & Documentation	Finalize project documentation	- Write the project report in Document. - Prepare user guide and technical documentation (API docs, Model cards).	10/11/2025 – 28/11/2025

Table 2.2. Project Planning and Execution Strategy

3. EXISTING SYSTEM / STATE OF THE ART

3.1 Overview of the Field

3.1.1 Action Recognition in Surveillance Video

Action recognition in video aims to automatically assign semantic labels such as *walking*, *fighting*, or *falling* to short segments of a video sequence. In surveillance, this problem appears in a stricter and more challenging form: video anomaly detection, where abnormal events (e.g., fire, violence, falls) are sparse and occur within long untrimmed streams dominated by normal activities. Only a small fraction of frames or segments are actually abnormal, which leads to severe class imbalance and makes evaluation and training more difficult [1], [3].

Traditional approaches often rely on hand-crafted features (e.g., optical-flow histograms, trajectories) combined with shallow classifiers. These methods struggle with complex scenes, camera motion, viewpoint variability, and occlusions that are common in CCTV environments [1], [2]. Modern deep-learning-based systems instead learn spatio-temporal representations directly from data and typically operate in one of two regimes:

- Unsupervised / weakly-supervised anomaly detection – Models such as autoencoders, prediction networks or one-class classifiers learn the statistics of normal behavior and flag deviations as anomalies [1], [3]. This reduces the need for dense annotations but often yields coarse, hard-to-interpret scores.
- Supervised multi-class action recognition – When labelled data is available, 3D CNNs or Transformer-based architectures are trained to predict action labels for clips, sometimes together with start–end timestamps in untrimmed videos [2], [4]. This offers more precise labels and interpretable outputs but requires careful dataset design.

For real-time surveillance, many works propose **integrated pipelines** that combine fast object detectors with spatio-temporal networks. A typical pattern is:

1. A **front-end detector** rapidly scans frames for people, fire regions, or other salient cues [5], [6].
2. Detected regions or buffered clips are passed to a **spatio-temporal backbone** (3D CNN, dual-pathway CNN, or video Transformer) to infer the underlying action [3], [4].
3. A **temporal aggregation head** merges clip-level predictions into segments with explicit start–end times, enabling timeline-based analysis and alerting.

The system in this project follows exactly this multi-stage design: YOLOv11 detectors identify candidate regions and time windows, SlowFast extracts clip-level features, MobileNetV3-Small verifies fire regions, and a lightweight temporal Transformer head converts clip predictions into interpretable incident segments.

3.1.2 CNN-based Video Models

Deep learning fundamentally transforms video understanding. Early CNN-based systems extended 2D image models to video by processing each frame independently and aggregating features over time via pooling or recurrent networks (e.g., LSTMs) [2]. Although simple and relatively efficient, these approaches often treat temporal structure as a secondary step and may miss fine-grained motion cues.

Subsequent research introduced **3D convolutional networks**, where convolution kernels span height, width, and time so that motion and appearance are modelled jointly. Representative families include C3D- and R(2+1)D-style networks. These architectures capture richer spatio-temporal patterns but are computationally expensive when applied naively to long surveillance videos [3].

Modern designs therefore emphasize multi-rate and multi-scale processing:

- Dual-pathway CNNs such as SlowFast operate a slow branch at low frame rate to capture high-resolution spatial semantics and a fast branch at higher frame rate to focus on rapid motion [4]. Lateral connections fuse information between the two pathways, achieving strong accuracy while keeping the fast branch relatively lightweight.
- CNN–Transformer hybrids run a CNN (2D or 3D) to produce clip-level features, then use Transformer layers to model long-range temporal dependencies and refine boundaries or action labels [4]. This hybrid strategy often offers a better accuracy–efficiency trade-off than using a full video Transformer end-to-end.

Overall, CNN-based video models remain a core component in many state-of-the-art action recognition and anomaly detection pipelines thanks to their strong performance and flexible integration with temporal modules [3], [4].

3.1.3 Summary

In summary, the state of the art in video understanding for surveillance combines several key ideas:

- CNN-based backbones (2D, 3D, and dual-pathway CNNs) provide strong spatio-temporal features for action recognition [3].
- Transformer-based modules extend the temporal receptive field and enable more flexible modeling of long-range dependencies, either as full backbones or as temporal heads. [4], [9].
- Hybrid pipelines that integrate detection, action recognition, and temporal localization are particularly well suited for intelligent surveillance, where abnormal events are rare, short, and embedded in long periods of normal activity [2], [5], [7], [8].

The architecture adopted in this project—YOLO11 for frame-level abnormality screening, SlowFast and MobileNet for spatio-temporal feature extraction—aligns closely with these trends and is designed to meet practical constraints of CCTV-based anomaly detection.

3.2 Key Studies and Theories

3.2.1 SlowFast Networks (Feature Extraction):

SlowFast was introduced as a dual-stream video architecture specifically designed to balance semantic richness and motion sensitivity in video recognition tasks [4]. The Slow pathway operates at a low frame rate (sampling one frame every several time steps), focusing on high-resolution spatial details such as object appearance and scene context. In contrast, the Fast pathway operates at a much higher frame rate but with reduced channel capacity, emphasizing rapid motion patterns and short-term dynamics [4].

Formally, if the Slow pathway samples frames with stride τ, the Fast pathway samples frames with stride τ/α, where $\alpha > 1$ controls the relative frame rate. The Fast pathway typically uses a fraction β of the channels of the Slow pathway, allowing the model to maintain high temporal resolution while keeping the overall computational cost manageable [4]. Features from both pathways are fused at multiple stages, enabling the network to jointly reason about what is happening (Slow) and how it is moving (Fast).

These design principles—multi-rate sampling and asymmetric channel allocation—are particularly attractive for surveillance scenarios, where abnormal events are rare, short, and embedded in long sequences of normal behavior. Semantics such as recognizing a person, a fire region, or a prone body position require detailed spatial cues, while abnormal behaviors such as fighting or falling involve fast, discriminative motion patterns that must be captured reliably [1], [3].

In this project, SlowFast R101 serves as the primary spatio-temporal feature extractor in Stage 2. YOLOv11 first detects persons and buffers 32/64 consecutive frames around these detections to form fixed-length clips. These clips are fed into SlowFast, which outputs high-level spatio-temporal embeddings and per-clip logits for the four classes (Fire, Fighting, Fall, Normal). This combination exploits SlowFast's strong clip-level representation while delegating long-range temporal reasoning to a lightweight Transformer.

3.2.2. MobileNetV3-Small

MobileNetV3-Small is a lightweight convolutional architecture designed for high-efficiency visual recognition tasks, making it particularly suitable for real-time CCTV analysis under limited GPU resources. It builds upon the foundations of MobileNetV1/V2 by incorporating depth wise-separable convolutions, inverted residual blocks, and Squeeze-and-Excitation (SE) attention mechanisms to recalibrate channel-wise features. In addition, MobileNetV3 introduces architecture search and hard-swish activations, significantly improving the accuracy–efficiency trade-off on embedded and edge devices [9].

Structurally, MobileNetV3-Small uses narrower layers, reduced channel widths, and a bottleneck-heavy design while still capturing essential spatial cues. This makes it effective for appearance-based classification tasks where fine-grained local features—such as flame texture, specular highlights, or color-intensity gradients—are important discriminators [9], [11]. As a result,

MobileNet-style networks are often adopted in fire-detection systems that must run in real time or on resource-constrained hardware [11], [13].

In this project, the MobileNetV3-Small functions as a fire-verification module (Stage 2B). After YOLOv11 flags a region as potentially containing fire, the cropped ROI (224 × 224, with padding) is normalized and passed through a MobileNetV3-Small backbone followed by a custom classifier head consisting of Linear → Hardswish → Dropout → Linear layers. This head outputs logits for the Fire / Non-Fire classes, enabling the system to distinguish real fire from visually similar distractors such as red clothing, orange lighting, or reflections. Only detections confirmed by MobileNetV3-Small are treated as valid fire events and forwarded to the temporal smoothing and alert generation module. Thanks to its compact design, this stage adds negligible computational overhead while significantly reducing false fire alarms [9], [11], [13].

3.2.3 YOLOv11

The “You Only Look Once” (YOLO) family of detectors introduced the paradigm of single-stage, real-time object detection, in which a neural network directly predicts bounding boxes and class probabilities from the entire image in a single forward pass [5]. Over successive versions, YOLO models have improved accuracy, robustness, and flexibility while preserving their hallmark speed advantage, making them a natural choice for CCTV-based applications [5], [6]. Recent comparative studies highlight next-generation variants such as YOLOv8–v11 as particularly strong baselines for safety-critical vision tasks [6], [7].

The proposed system adopts YOLOv11 as its front-end object detector for both persons and fire cues. Although YOLOv11 is trained primarily for generic object detection, its architecture and training regime make it highly effective at:

- Detecting people, which serve as the primary actors in actions such as fighting and falling.
- Detecting visual cues such as fire regions, smoke, or objects indicative of dangerous situations.
- Operating at high frame rates, enabling near real-time processing of video streams [5], [7].

Within the pipeline, YOLOv11 plays several critical roles. First, Stage 1 – Person Detection filters frames to those containing humans and buffers 32/64 consecutive frames around them to build clips for the SlowFast backbone. This reduces the number of irrelevant clips and focuses Stage 2 on human-centred activity. Second, Stage 1 – Fire Detection provides fire-centred crops with padding, which are sent to the MobileNetV3-Small fire-verification module (Stage 2B). By providing precise bounding boxes, YOLOv11 reduces background noise and improves the quality of the inputs for both the spatio-temporal backbone and the fire-verification branch [1], [5], [6].

By combining YOLOv11 with SlowFast R101, the system retains the speed and localization strength of YOLO while gaining powerful temporal modelling capabilities downstream [1], [4], [20]. YOLOv11 handles dense, frame-level detection; SlowFast extracts rich clip-level spatio-temporal features; and the temporal Transformer aggregates these features into coherent incident segments with estimated start–end times for Fire, Fighting, and Fall events.

3.3 Justification for the Project Architecture

The choice of combining YOLOv11, MobileNetV3-Small, SlowFast R101, and a multi-scale temporal Transformer head is not arbitrary. It is grounded in the characteristics of CCTV anomaly detection—long untrimmed videos, sparse abnormal events, static camera viewpoints—and in the practical constraints of running on a single RTX 3060 GPU. Recent surveys on video anomaly detection emphasise that scalable systems must (i) efficiently screen large volumes of mostly normal footage, (ii) model fine-grained spatio-temporal patterns for rare events, and (iii) expose results in an interpretable way for human operators [1], [3]. The adopted architecture is designed explicitly around these requirements.

Alignment with surveillance constraints

Surveillance footage in real environments typically consists of hours of normal activities with only a few short abnormal segments, making exhaustive 3D processing of every frame impractical [1]. In the proposed system, Stage 1 (YOLOv11) filters frames by detecting people and fire cues, and only buffers 32/64-frame clips around those detections. Fire regions are further validated by MobileNetV3-Small before being treated as true fire events. This design greatly reduces the number of clips that must be processed by the heavier video backbone while preserving high recall for potentially dangerous situations. YOLO-style detectors are particularly suitable for this front-end role because they offer a strong balance between accuracy and speed, enabling near real-time processing of CCTV streams on mid-range GPUs [5], [6].

Strong empirical foundations

Each component in the pipeline is supported by strong empirical evidence from the literature. YOLO-based detectors have repeatedly demonstrated state-of-the-art trade-offs for real-time object detection, and next-generation variants such as YOLOv11 are widely adopted in safety-critical applications [5], [7]. SlowFast Networks have achieved excellent performance on large-scale video benchmarks such as Kinetics and AVA by jointly modelling slow semantic changes and fast motion details, making them well suited for actions like fighting or falling [4]. Lightweight MobileNet-style architectures, including MobileNetV3-Small, are a standard choice for edge classification tasks and have been successfully applied to video fire detection, where computational budgets are tight but appearance cues such as flame texture and color are crucial [9], [11], [13]. Finally, recent work on transformer-based temporal action localisation shows that compact temporal Transformer heads can effectively aggregate clip-level features into precise action segments, motivating the Stage-3 multi-scale Transformer design adopted in this project [20].

Modularity and extensibility

The architecture is intentionally modular. Stage 1 (YOLOv11 + MobileNetV3-Small) encapsulates all frame-level detection and fire verification, Stage 2 (SlowFast R101) focuses on clip-level spatio-temporal representation. This separation allows individual components to be upgraded without redesigning the entire system—for example, replacing YOLOv11 with a newer detector, swapping

MobileNetV3-Small for another lightweight classifier, or experimenting with alternative temporal heads—while preserving the overall pipeline and data interfaces.

Balanced efficiency–accuracy trade-off

Finally, the architecture offers a balanced efficiency–accuracy trade-off appropriate for a prototype deployed on a single RTX 3060 GPU. YOLOv11 aggressively prunes obviously normal frames and provides spatially focused crops, reducing the workload for downstream modules. MobileNetV3-Small adds a low-cost verification step that significantly decreases false fire alarms. SlowFast R101, while more computationally demanding, processes only selected clips and thus remains feasible in practice, especially when combined with mixed-precision training and inference [4], [5]. The temporal Transformer head is deliberately lightweight compared to full video transformers, yet it adds the capability to predict coherent incident segments with start and end timestamps. Taken together, these considerations justify the selected architecture as a practical and theoretically well-motivated solution for abnormal behaviour detection and localisation in surveillance video.

3.4. Benchmark Comparison

To further justify the adopted architecture, this section conceptually compares the proposed hybrid pipeline against two representative alternatives commonly discussed in video anomaly detection research: (i) single-stream 3D CNN backbones without detection, and (ii) detector-only pipelines that rely solely on frame-level object detectors such as YOLO. Recent surveys highlight that these families capture the main design choices in current systems for CCTV anomaly detection [1], [3]. Table 3.4 (Benchmark Comparison) in the Appendix summarizes this comparison along the dimensions of accuracy, temporal precision, and computational cost.

Single-stream 3D CNN baseline (no detector).
A straightforward approach is to feed resized frames or short clips directly into a 3D CNN backbone (e.g., a Slow-only or C3D-style model) and use the network's logits to classify each clip as normal or abnormal [1], [4]. This design benefits from end-to-end spatio-temporal modelling and can achieve strong recognition accuracy on trimmed benchmark datasets. However, when applied to long untrimmed surveillance videos, it must process *every* frame or clip, leading to high computational cost and latency. Furthermore, without an explicit detector, the model may waste capacity on background regions and struggle with small or partially occluded actors, both of which are frequent in CCTV footage [1], [17].

Detector-only baseline (YOLO-style).
At the other extreme, one can rely on a fast object detector (e.g., YOLO variants) as a frame-level abnormality indicator, triggering alarms directly from detected objects such as fire regions or human poses [5], [6]. Detector-only pipelines excel at speed and are simple to deploy: they can run at real-time frame rates even on modest hardware, and they provide interpretable bounding boxes around suspicious regions [5], [7]. However, they lack explicit modelling of temporal dynamics. Without analysing motion patterns over time, such systems cannot reliably differentiate normal vs. abnormal human interactions (e.g., friendly hugging vs. aggressive fighting) or transient

poses that resemble falls but quickly recover. As a result, detector-only baselines often exhibit high false positive and false negative rates for complex behaviours.

Proposed hybrid YOLOv11 + MobileNetV3-Small + SlowFast + Temporal Transformer.

The architecture in this project combines the strengths of both worlds while mitigating their weaknesses. YOLOv11 serves as a front-end filter that quickly scans the video, detects persons and candidate fire regions, and constructs 32/64-frame clips around these events. A lightweight MobileNetV3-Small fire-verification branch further filters fire detections, suppressing false alarms caused by red clothing or warm lighting [9], [11], [13]. Only these focused clips are passed to SlowFast R101, which provides rich spatio-temporal features that capture both slow semantic evolution and fast motion changes [4]. Finally, a multi-scale temporal Transformer head aggregates clip-level predictions into coherent incident segments with start–end timestamps, improving temporal precision without the cost of a full video transformer backbone [20].

Compared to the 3D-only baseline, the proposed pipeline avoids processing obviously normal content by leveraging YOLOv11 pre-filtering, achieving better scalability on long CCTV streams. Compared to the detector-only baseline, the inclusion of SlowFast and the temporal Transformer head enables fine-grained reasoning over motion and temporal context, which is essential for distinguishing fighting, falling, and normal actions. This benchmark-style comparison supports the choice of hybrid architecture as a balanced solution in terms of accuracy, temporal localisation quality, and computational efficiency for practical surveillance deployment [1], [3], [4].

4. METHODOLOGY

The development of the AI-Based Abnormal Behavior Detection System follows a methodology that tightly couples research questions, model choices, and deployment constraints. Instead of a single monolithic network, the system adopts a three-stage hybrid pipeline:

- Stage 1 – Detection & Preprocessing: YOLOv11 detects persons and fire-like regions in CCTV frames and produces spatially focused regions of interest (ROIs). A MobileNetV3-Small classifier refines fire predictions and filters out red clothing, lighting artifacts, and other fire-like distractors. [5], [6], [9]
- Stage 2 – Spatio-Temporal Feature Extraction: person-centered clips buffered around YOLOv11 detections are passed to a SlowFast R101 backbone, which learns rich spatio-temporal representations for Normal, Fighting, and Fall behaviours and, when needed, fire context. [3], [4]
- Output– Temporal Localization and Alert Aggregation: sequences of per-clip features and fire events are processed by a lightweight multi-scale temporal Transformer encoder, which outputs labelled incident segments with start–end timestamps. These segments are serialized as JSON alerts and displayed on a Flask dashboard. [1].

This design aims to satisfy two conflicting requirements highlighted in the anomaly-detection literature: accurate temporal localization of rare events and efficient processing of long, mostly normal surveillance streams. [1], [2]

4.1 Research Questions and Objectives

4.1.1 Research Questions

The project addresses four main research questions in deploying deep learning for abnormal behaviour detection in untrimmed CCTV videos:

- **RQ1 – Efficient candidate selection.**
  How can a fast detector such as YOLOv11 be used to filter out obviously normal content so that heavy 3D CNN and Transformer modules operate only on informative regions? The goal is to avoid dense 3D processing of long videos while maintaining high recall for rare anomalies. [1], [5]
- **RQ2 – Spatio-temporal representation and temporal boundaries.**
  What combination of a dual-pathway 3D CNN backbone (SlowFast R101) and a compact temporal Transformer head yields both strong action classification performance and precise start–end localization of Fire, Fighting, and Fall events in long untrimmed videos? Recent Transformer-based temporal action localization (TAL) work suggests that self-attention over multi-scale temporal features is particularly effective in this setting. [3].
- **RQ3 – Handling data imbalance and rare anomalies.**
  How can training strategies such as class-balanced sampling, oversampling of rare classes, and targeted data augmentation improve robustness when abnormal classes (Fighting, Fall,

Fire) are much rarer than Normal activity in both public datasets and custom recordings? [2], [10], [17]

- **RQ4 – Bridging prototype and deployment.**
  How can a research-grade model be packaged into a usable system with acceptable latency, resource usage, and interpretability for non-expert operators? This question covers GPU memory constraints, throughput on an RTX 3060, and a dashboard that presents bounding boxes and timelines instead of raw tensors. [3], [5], [8]

4.1.2 Technical Objectives

To answer these questions, the methodology defines the following technical objectives:

1. Design a three-stage hybrid pipeline that combines YOLOv11 person/fire detection, MobileNetV3-Small fire verification, a SlowFast R101 backbone for abnormal-action recognition, and a multi-scale temporal Transformer head for incident segmentation. [3], [4], [6]
2. Construct an aggregated anomaly dataset from public benchmarks (e.g., UCF-Crime, violence datasets) and custom campus-like recordings. All videos are relabelled into the four target classes—Fire, Fighting, Fall, Normal—with temporal boundaries for abnormal segments to support localization. [1], [2], [7]
3. Fine-tune backbone models:
   a. Fine-tune SlowFast R101 on 32-frame person-centered clips for 4-class clip-level classification. [3], [4]
   b. Fine-tune MobileNetV3-Small as a lightweight binary fire vs. non-fire classifier on ROIs cropped by YOLOv11. [9], [11]
4. Integrate the pipeline into a Flask backend and web dashboard that:
   a. accepts user-uploaded CCTV videos,
   b. runs the full pipeline with GPU acceleration on an RTX 3060.
   c. exposes JSON alerts via a REST API, and
   d. visualizes incidents as a colour-coded timeline synchronized with video playback. [3], [5], [8].

4.1.3 Abnormality Detection Layer (YOLO11)

Stage 1 acts as a **fast abnormality trigger** and prepares structured inputs for the later stages. Instead of processing every frame with a 3D CNN, the system leverages YOLOv11 to highlight potentially dangerous regions and then refines fire decisions with a dedicated MobileNetV3-Small classifier. [5], [6], [9]

a. YOLOv11 person and fire detection

Each input video is decoded into frames at a fixed frame rate (typically 10–15 FPS for long CCTV streams). Each frame is passed through YOLOv11, which outputs bounding boxes, class labels, and confidence scores for:

- Person objects used to construct human-centered action clips.
- Fire / smoke-like regions, used as candidate for fire anomalies. [6], [7], [8]

YOLOv11 builds on the CSPDarknet backbone with C3k2 blocks, FPN+PAN neck, and decoupled detection heads, offering an excellent speed–accuracy trade-off for real-time safety-critical vision tasks. [6], [7] For person detection, the system uses a YOLOv11-Pose variant, which additionally predicts human keypoints. Keypoints provide geometric context (e.g., head vs. hip position) that is useful for distinguishing falls from non-dangerous bending motions and for cropping tight person ROIs.

b. Fire ROI extraction and verification

For every frame where YOLOv11 detects a fire-like region, the detector's bounding box is expanded with a small padding ratio (e.g., 10–15%) and cropped to a fixed resolution (224 × 224). These crops are routed through a two-step fire verification branch:

1. **Initial fire / non-fire classification:**
   A YOLOv11-based fire classifier produces a preliminary probability that the ROI corresponds to fire. This leverages the same architecture but re-trained on a curated fire vs. non-fire dataset, benefiting from YOLOv11's strong feature extraction capabilities for amorphous objects such as flames and smoke. [8], [11]
2. **MobileNetV3-Small verification:**
   Crops that pass a low confidence threshold are then fed to a MobileNetV3-Small backbone followed by a custom classifier head (Linear → Hardswish → Dropout → Linear), as illustrated in the MobileNet pipeline diagram.
   MobileNetV3-Small combines inverted residual blocks with squeeze-and-excitation attention and hard-swish activations, providing robust appearance-based discrimination with very low latency—ideal for confirming fire vs. non-fire on many small ROIs. [9], [11], [13]

Only detections confirmed as fire by the MobileNet classifier are promoted to confirmed fire events. This cascaded design significantly reduces false alarms from red clothing, traffic lights, reflections, or sunset lighting while keeping computational overhead small. [11], [12], [14]

c. Person-based abnormality triggers

For person detections, the system computes a frame-level abnormality score that considers:

- the number of people in close proximity,
- unusual poses inferred from YOLOv11 keypoints (e.g., head close to the ground),
- sudden changes in person positions across consecutive frames. [1], [3], [17]

Frames whose scores exceed a configurable threshold are marked as triggers for action-clip extraction. This follows the design pattern where a fast detector is used as a front-end filter to focus 3D processing on promising regions. [1], [2], [6]

d. Temporal smoothing of fire predictions

Fire predictions are further stabilised using temporal smoothing over 3–5 neighbouring frames. A short moving window aggregates fire/non-fire logits from consecutive frames and suppresses isolated one-frame spikes caused by transient lighting changes or compression artifacts. Smoothing in this way is standard practice in video fire-detection systems to balance responsiveness and robustness. [11], [13]

4.1.4 Clip Generation and Buffering

Stage 1 triggers must be converted into temporal clips compatible with the SlowFast backbone. The clip-generation strategy seeks to capture enough context around anomalies without exploding computational cost. [2], [3]

a. Trigger-centred sliding windows

For each abnormal person-frame at t , the system constructs a trigger-centred window of length L frames by including *Δpre* frames before and Δ_{post} frames after the trigger:

$$L = \Delta_{pre} + 1 + \Delta_{post}$$

In practice, we set $L = 32$ with

$$\Delta_{pre} = \Delta_{post} = 16$$

Which matches the 32-frame input configuration used by SlowFast R101 [3], [4]. This window captures both the build-up to an event (e.g., approaching opponents before a fight) and its immediate aftermath (e.g., lying on the floor after a fall).

b. Overlap control and redundancy reduction

If multiple triggers occur in close temporal proximity, their windows may overlap. To avoid generating many nearly identical clips, the system:

- merges candidate windows whose temporal Intersection-over-Union (tIoU) exceeds a predefined threshold (e.g., 0.7), and
- enforces a maximum number of clips per video by discarding very low-score triggers when the budget is exceeded.

This strategy is analogous to non-maximum suppression in temporal action detection and is widely used to limit redundancy while preserving coverage. [1].

c. Sliding buffer implementation

A sliding frame buffer maintains the most recent L frames for each video stream. When a new trigger arrives, the system can immediately assemble the pre-context from the buffer without re-decoding older portions of the video. For offline processing, the buffer resides in main memory; for live CCTV streams, it is aligned with real-time decoding.

d. Supervised clip labelling

For training, each clip is assigned a label based on its overlap with ground-truth segments:

- If the clip overlaps sufficiently (e.g., tIoU ≥ 0.5) with a labelled Fire, Fighting, or Fall interval, it inherits that label.
- Clips with no significant overlap with any abnormal segment are labelled Normal.

This clip-level supervision is used to fine-tune SlowFast as a 4-class classifier and to provide target labels for training the temporal Transformer head. [2], [7]

4.1.5 Spatio-Temporal Modeling Layer

Stage 2 performs the core video understanding. Each clip produced in Stage 1 is forwarded to a pre-trained and fine-tuned SlowFast R101 network. [3], [4]

a. Dual-pathway feature extraction

SlowFast employs two parallel pathways:

- Slow pathway: samples frames sparsely (e.g., every 4th frame) and uses a high channel capacity to capture static appearance and long-term semantics (scene layout, person identity, presence of fire in the scene).
- Fast pathway: samples frames at a much higher rate (e.g., 8× faster) but with fewer channels, focusing on rapid motion dynamics such as punches, kicks, or the downward acceleration of a fall. [4]

Lateral connections inject Fast-path features into corresponding Slow-path stages, allowing the model to jointly reason about “what is present” and “how it moves”. This dual-rate mechanism has been shown to work particularly well for actions defined by motion intensity and duration. [3], [4]

The output for each 32-frame clip is a high-dimensional feature vector (dimension$D$, e.g., 2048–2304) summarizing both spatial and temporal information. Over a full video, the model produces a sequence of such vectors, one per overlapping clip, which forms the input to the temporal Transformer head.

b. Clip-level classification head

On top of the pooled SlowFast features, a linear classifier maps each vector to logits over the four classes {Normal, Fighting, Fall, Fire}. This head is used both:

- during training, to optimize clip-level cross-entropy loss, and
- during evaluation, to compute clip-based accuracy and F1-scores for anomaly detection. [3], [7]

Clip-level predictions also act as strong priors for the subsequent temporal Transformer, which refines boundaries and aggregates neighboring clips into segments.

c. Integration with the fire branch

Although confirmed fire ROIs are primarily handled by the MobileNetV3-Small branch, SlowFast also receives clips that contain prominent fire regions (e.g., when flames occupy a large part of the scene). This allows the backbone to learn contextual cues such as people near fire or fire combined with panic movement, which distinguishes hazardous fire events from benign controlled flames (e.g., in a video screen). [11], [13]

4.1.6 Post-Processing and Visualization

Stage 3 (Output) converts the raw per-clip classification scores from the upstream branches (SlowFast and MobileNetV3) into interpretable incident alerts that can be visualized on the dashboard. Instead of using complex temporal regression models, this stage employs a lightweight, logic-based decision module to ensure real-time responsiveness.

a. Logic-Based Decision Module

The stream of predictions is processed by a Decision Module that acts as the final filter before alerting. This module operates on the probability distributions output by the classification heads in Stage 2.

Key design elements:

- Confidence Thresholding: To minimize false alarms, raw logits are converted into probabilities (via Softmax for actions or Sigmoid for fire). An alert is only triggered if the maximum class probability exceeds a strictly defined threshold (e.g., t > 0.75$). This filters out uncertain predictions often caused by transition frames or ambiguous movements.
- Temporal Stamping: Since the system processes fixed-length clips (e.g., 64 frames), the module maps the index of the detected clip back to the original video's timeline. This

assigns a precise Timestamp (Current Time or Video Time) to the detected event without needing regression.

- Event De-bouncing (Optional): To prevent alert flooding (e.g., sending 10 alerts for 10 consecutive frames of the same fire), the module implements a simple suppression logic that groups consecutive positive detections into a single continuous "Event" until the signal drops below the threshold.

b. JSON Serialization and API Exposure

Once an anomaly is confirmed by the Decision Module, it is serialized into a structured format for the frontend.

- Data Structure: Each alert is formatted as a standardized JSON object containing:
  - `track_id`: Identifier for the tracked person (from YOLOv11).
  - `event_type`: The classified label (Fire, Fall, Fighting).
  - `confidence`: The probability score associated with the prediction.
  - `timestamp`: The exact time the event occurred.
  - `snapshot_path`: A reference to the saved frame/clip for review.

These JSON objects are immediately pushed to the Flask REST API, triggering the visual update on the dashboard's timeline and alert panel. This direct-output approach ensures minimal latency, making the system suitable for live monitoring applications.

4.2 Data Collection and Preprocessing

The project utilizes an aggregated dataset focused on abnormal and violent actions, prepared through a structured pipeline to optimize training.

Step	Description	Detail / Configuration
Frame Extraction	Convert raw video into image sequences for uniform processing.	Use cv2.VideoCapture()/ffmpeg; fixed 5–10 FPS.
Normalization	Standardize input dimensions and pixel values.	Resize to 224×224; pixel normalization using ImageNet mean/std.
Clip Generation	Slice frame sequences into clips for motion modeling.	32 frames per clip; stride 16 (50% overlap) for temporal continuity.
Labeling	Assign class labels and temporal boundaries for training.	Each clip is labeled (Fire/Fighting/Fall/Normal).

Oversampling	Balance class distribution, addressing rarity of events.	"Hard cases" (Fire, Fighting, Fall) were oversampled 3× to balance difficulty.
Data Split	Define training, validation, and test sets.	70% training, 15% validation, 15% test.

Table 4.2 Data Collection and Preprocessing

4.2.1. Anomaly Detection Dataset

Class	Training	Validation	Testing	Total Sequences
Normal	133	31	31	**195**
Fall	59	3	3	**65**
Fighting	118	35	31	**184**
Fire	5,399	1,312	1,329	**8,040**
Total	**5,710**	**1,381**	**1,394**	**8,485**

Table 4.2.1. Dataset Distribution by Class and Split

The anomaly dataset is built from a mixture of public surveillance benchmarks (e.g. UCF-Crime, XD-Violence) and custom-recorded campus-style footage. This combination provides both the diversity of large-scale benchmarks and the realism of the target deployment environment, as recommended in recent surveys on video anomaly, fall, and violence detection.

To better reflect the structure of the pipeline, the dataset is explicitly decomposed into two parts:

a. Fire Verification Sub-Dataset (Frame-Level)

- Positive samples (Fire): frames where clear flames or dense smoke are visible. These are collected from fire-detection research datasets and curated online videos, following the characteristics discussed in recent fire-detection surveys and benchmarks (e.g., strong colour gradients, dynamic flame boundaries, smoke patterns).
- Hard negative samples (Non-Fire): visually similar but harmless phenomena such as red or orange clothing, traffic lights, neon signs, warm indoor lighting, reflections, or LED boards. Including these cases is crucial, because prior work shows that colour-only models often confuse them with true fire.
- Each sample is stored as a *224×224* RGB crop centred on the YOLOv11 fire-like bounding box, and labelled as Fire or Non-Fire for binary classification by MobileNetV3-Small.

b. Human-Action Sub-Dataset (Clip-Level)

- Normal: walking, standing, sitting, talking, or other benign activities.

- Fighting: pushing, punching, kicking, or close-range violent interactions involving one or more persons.
- Fall: sudden loss of balance followed by a downward motion and a lying-on-the-ground phase, consistent with clinical definitions in fall-detection literature.
- For each raw video, YOLOv11 is first applied to detect people. Temporal buffers around those detections (64 frames) are then cropped to obtain person-centred clips that serve as input to SlowFast. Manual inspection ensures that each clip contains a single dominant behaviour corresponding to one of the three classes.

In both sub-datasets, labels are stored in a unified JSON-style format (video id, start time, end time, class).

4.2.2 Data Preprocessing Methodology

Although the fire-verification and human-action branches share several preprocessing steps, there are also branch-specific operations tailored to their roles in the pipeline.

a. Frame Extraction and Basic Normalization

- Decoding: raw CCTV videos are decoded using cv2.VideoCapture / ffmpeg. To reduce redundancy while preserving motion, a fixed frame rate (typically 10–15 FPS) is used for offline dataset generation.
- Resizing: all frames and crops are resized to *224×224* before being fed into either MobileNetV3-Small or the SlowFast preprocessing pipeline, matching common practice for ImageNet-pretrained backbones.
- Normalization: pixel values are normalized using the standard ImageNet mean and standard deviation for each channel to stabilize training and leverage pre-training benefits.

b. Fire-Verification Branch (MobileNetV3-Small)

For the fire/non-fire module, the preprocessing is intentionally purely spatial, because the model operates on independent frames/crops:

YOLOv11-based ROI generation:

- YOLOv11 is applied frame-by-frame to detect fire-like regions. Bounding boxes with sufficient confidence are cropped with a small padding margin to preserve context (e.g., surrounding smoke or reflections).

Colour-aware preprocessing:

- Crops are converted to RGB, resized, and normalized. No temporal resampling is needed, since each crop is treated as an independent sample for MobileNetV3-Small.

Label attachment:

- During dataset construction, each crop is labelled **Fire** or **Non-Fire** based on manual annotation; these labels are later used for supervised training of the binary classifier. The design follows recommendations from fire-detection surveys that stress the importance of including hard negatives and appearance cues rather than relying only on motion.

c. Human-Action Branch (SlowFast)

For the SlowFast backbone and the temporal Transformer head, preprocessing preserves temporal structure while focusing on person-centred information:

Person detection and tracking:

- YOLOv11 detects people in each frame. Detections are linked across time using a lightweight IoU-based tracker, producing short tracks for each individual. This design is consistent with the Stage-1 description where YOLOv11 acts as a filter and ROI generator for downstream modules.

Clip generation:

- For each track, fixed-length clips of 64 frames are generated using a sliding window with 50% overlap. The window is centred on moments where abnormal postures or interactions are likely (e.g., sudden falls, rapid movements, close-range contact), ensuring that the resulting clips contain sufficient temporal context for SlowFast.

Spatial cropping and resizing:

- Each frame in a clip is cropped to the corresponding person bounding box (with padding), then resized to *224×224*. The clips are stored in a tensor layout compatible with SlowFast (e.g., T×H×W×C → C×T×H×W).

Label assignment for temporal training:

- For segments where the entire 64-frame window corresponds to a single behaviour, the clip is given a single class label (Normal/Fighting/Fall).
- For longer events spanning multiple clips, all overlapping clips are assigned the same label and their start–end times are recorded. These annotation sequences are later fed into the temporal Transformer head to learn segment-level aggregation.

d. Dataset Split

Both branches follow the same split protocol: 70% Training, 15% Validation, 15% Test, with videos, not frames, as the unit of splitting. This prevents information leakage between splits (e.g., consecutive frames from the same video appearing in both training and test sets) and follows best practices in anomaly-detection research.

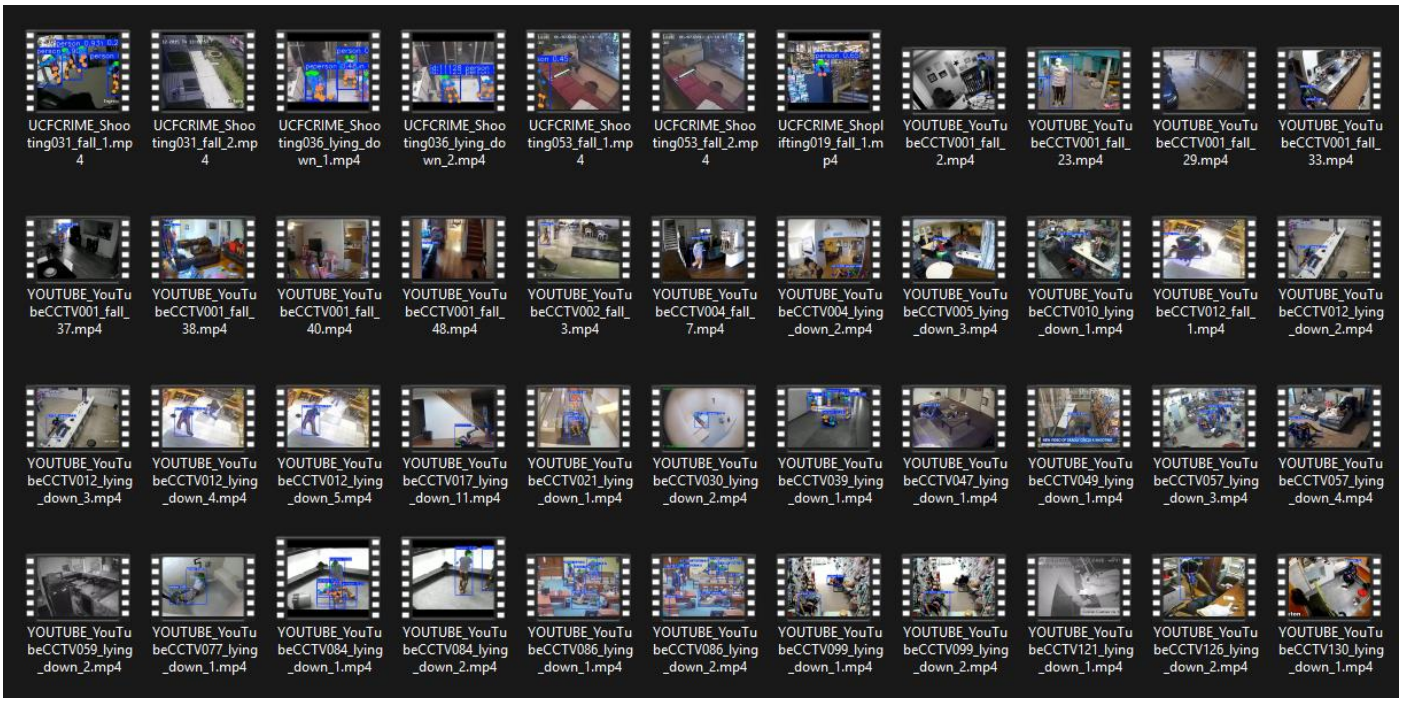

Figure 4.2.2. Sample Data with Bounding Box Placement for Pose Detection

4.2.3 Dataset Statistics and Distribution

The final dataset exhibits the typical class imbalance observed in surveillance anomaly detection: Normal activity and Fire events are much more frequent than clear Fighting or Fall incidents. This mirrors findings from existing anomaly, fall, and violence datasets, where critical events occupy only a small fraction of the total duration.

To quantify this, the project reports statistics (see Table 4.2.3 in the report) along three axes:

- Number of samples per class: counts of fire/non-fire crops for the MobileNet branch and of Normal/Fighting/Fall clips for the SlowFast branch.
- Temporal coverage: the total number of frames and approximate video duration contributed by each class.
- Average clip length and variance: confirming that generated clips are long enough to capture complete falls or fights but still short enough for efficient training and inference.

Fire samples dominate the frame-level dataset because fire scenes are densely annotated by frame-by-frame, whereas fighting and falls occur in shorter bursts. Conversely, in the clip-level dataset, Normal clips remain in the majority class due to long stretches of benign behavior, while Fighting and Fall clips are rare but critical. This motivates the focused augmentation and sampling strategies described next.

Characteristic	Total Sequences	Total Frames	Avg. Frames/Sequence	% of Total Sequences
Overall Total	**12,869**	**162,300**	**12.6**	**100.0%**
Normal	272	27,401	100.7	2.1%
Fall	180	55,271	**307.1**	1.4%
Fighting	4,377	63,269	14.5	34.0%
Fire	8,040	16,359	2.0	**62.5%**

Table 4.2.3. Overall Dataset Distribution and Imbalance Analysis

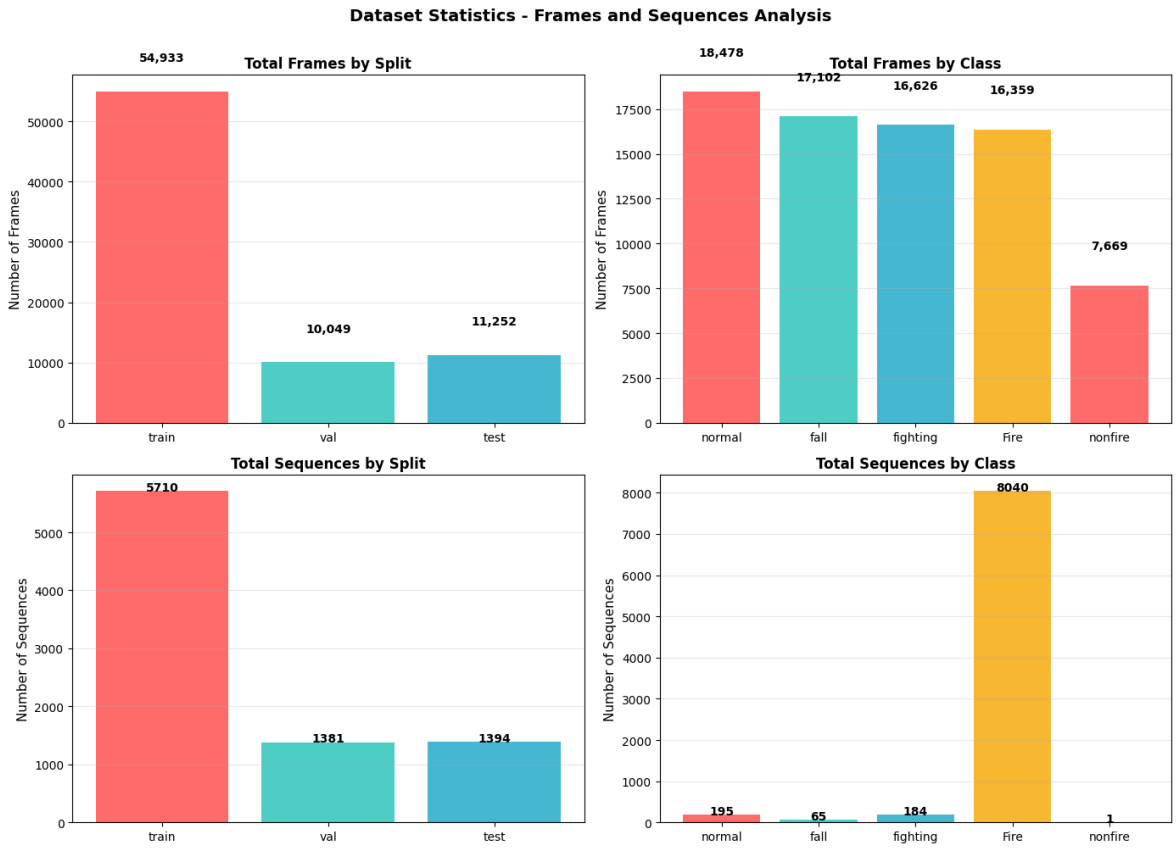

Figure 4.2.3 Frames and Sequences Analysis

4.2.4 Data Augmentation and Sampling Strategy

To address RQ3 on data imbalance and robustness to real-world noise, the training pipeline applies a combination of spatial augmentations, a dedicated fire/non-fire verification branch, and class-balanced sampling. These strategies are applied consistently across the SlowFast backbone and the MobileNetV3-Small fire-verification module, while YOLOv11 retains its standard detector-level augmentations from the official training recipe [2], [3], [5], [7].

a. Spatial and Photometric Augmentation for Person-Centred Clips

For the SlowFast backbone, each training sample is a fixed-length RGB clip (32 or 64 frames) constructed from the pre-buffered person tracks described in Section 4.2.2. To make the model robust to variations in camera viewpoint, lighting, and partial occlusion, a set of spatial and photometric augmentations is applied online to every mini-batch [3], [4], [17]:

- Random horizontal flip (p = 0.5): Simulates left–right symmetry in scenes such as corridors or staircases. This prevents the network from overfitting a particular camera orientation (e.g., fights always occurring on the “left-hand side” of the frame).
- Random scaling and cropping: Input frames are first resized and then randomly cropped to 224 × 224. This exposes the model to people appearing at different sizes and positions, which is important when cameras are installed at different heights or zoom levels.
- Random Erasing / cut-out style occlusion: Small rectangular regions of the frame are replaced with a constant value or noise to mimic occlusions from pillars, furniture, or passing pedestrians. This forces SlowFast to rely on global motion patterns rather than a single discriminative patch when deciding between Normal, Fighting, and Fall.
- Colour jitter (brightness/contrast/saturation shifts): Mild perturbations in colour channels simulate different lighting conditions (day vs. night, indoor vs. outdoor, warm vs. cool white balance). This makes the model less sensitive to changes in exposure or camera calibration.

Because these operations are applied independently to each clip instance, the same underlying event can appear in many augmented variants across epochs. This effectively multiplies the size of the training set without collecting more raw footage, and improves generalisation to new CCTV cameras that were not seen during data collection [3], [4].

b. Dedicated Fire / Non-Fire Verification Branch with Targeted Augmentation

Even with a strong detector such as YOLOv11, fire-like regions remain challenging: many everyday objects (red clothing, traffic lights, decorative lamps, LED screens, warm reflections) share similar colour and texture patterns with real flames. To increase overall precision and reduce spurious fire alerts, the system introduces a separate verification stage based on MobileNetV3-Small (Stage 2B) [9], [11], [13].

During training, a dedicated fire-verification dataset is built from YOLOv11 crops and additional non-fire distractor images:

- Positive samples (Fire): Cropped regions where visible flames or strong fire patterns are present.
- Negative samples (Non-Fire): Crops containing confusing but harmless content such as red clothes, orange lighting, reflections, traffic lights, or bright signage.

These 224 × 224 crops are passed through the MobileNetV3-Small backbone with a custom Linear → Hardswish → Dropout → Linear classifier head, as described in Section 3.2.2. To make this binary classifier robust, the following augmentations are applied:

- Strong colour jitter and contrast changes: Emphasise that the model must focus on flame structure and texture rather than raw RGB intensity. For example, a red T-shirt under bright light should still be classified as Non-Fire even if its pixel histogram resembles a flame patch.
- Random rotations and slight perspective transforms: Account for the fact that fire alarms may be triggered from tilted or off-axis views (e.g., ceiling cameras looking down at a stove or trash bin).
- Random Erasing and small random crops: Simulate partial visibility of fire (e.g., flames appearing only at the edge of a trash can or behind an object) and encourage the network to detect fire even when only part of the flame is visible.

From a pipeline perspective, this additional training step is crucial: YOLOv11 first casts a wide net to retrieve any region that might contain fire, prioritizing recall. MobileNetV3-Small then refines these candidates into Fire vs. Non-Fire with a much stricter decision boundary. As a result, the number of false fire alarms entering the temporal smoothing and alert-generation stage is significantly reduced, and the overall system achieves higher precision without increasing the computational cost of Stage 3 [9], [11], [13].

c. Class-Balanced Sampling for Rare Anomalies

The aggregated dataset is highly imbalanced: Normal clips dominate, while Fire, Fighting, and Fall events are rare and often short. Training directly on this distribution would lead the models to favour the Normal class, harming recall on the anomalies that matter most in practice [2], [10], [17]. To counter this, the training pipeline adopts class-balanced sampling strategies at both the clip level (SlowFast) and the crop level (MobileNetV3-Small):

- Balanced mini-batches for SlowFast: Each mini-batch is constructed such that rare classes (Fire-context clips, Fighting, Fall) are intentionally oversampled relative to their raw frequency. In implementation, a weighted sampling scheme is used where the probability of drawing a clip from class *ccc* is inversely proportional to its dataset frequency. This ensures that, over an epoch, the network sees a comparable number of examples from each class and learns discriminative features for all three anomaly types.
- Balanced Fire vs. Non-Fire crops for MobileNet: Similarly, the fire-verification dataset is sampled so that Fire and Non-Fire crops appear in roughly equal proportions per mini-batch. This prevents the MobileNet branch from collapsing to a trivial “mostly Non-Fire” solution and encourages it to model the subtle visual differences between real flames and look-alike objects.
- Hard-example emphasis: Misclassified or low-confidence samples (for example, borderline interactions between friendly play and Fighting, or very small Fire patches) can be revisited more frequently during later epochs. This implicit curriculum focuses learning on the most ambiguous regions of the feature space, improving robustness near the decision boundaries.

Together, these augmentation and sampling strategies operationalise the training objectives outlined in Section 4.1.2 for handling data imbalance and real-world noise. They allow the models

to maintain high sensitivity to rare Fire, Fighting, and Fall events while keeping false positives under control, which is essential for a practical surveillance system where operators must trust the alerts they receive [2], [10], [17].

Component	Technique/Implementation	Rationale
Mixing Augmentation	Mixup (Enabled) & CutMix (Enabled)	Mixup/CutMix handles regularization through data space mixing, creating composite samples to improve generalization and decision boundary smoothness.
Occlusion/ Noise	Random Erase (Enabled)	Simulates partial occlusion or momentary loss of visibility, strengthening the model's ability to classify actions even with incomplete spatial information.
Spatial Robustness	Random Horizontal Flip (Enabled, ρ =0.5)	Maintains orientation robustness, ensuring the model is invariant to the direction of movement or action observed by the camera.

Table 4.2.4. Configuration of Temporal Sampling and Training Augmentations

4.3 Model Architecture

Building upon the pipeline outlined in Chapter 1, the final system architecture establishes a streamlined two-stage workflow comprising a detection-and-filtering front-end and specialized recognition of backbones. Stage 1 (YOLOv11 + MobileNetV3-Small) operates directly on CCTV frames to detect people and fire-like regions, acting as a fast gatekeeper that forwards only potentially abnormal content to heavier models. Stage 2 (SlowFast R101) encodes human-centered clips into high-level spatio-temporal features and produces direct clip-level predictions for Normal, Fighting, and Fall behaviors. Instead of utilizing a complex temporal regression head, the system employs a direct inference mechanism where high-confidence predictions are immediately converted into JSON alerts and exposed via a Flask REST API for visualization on the dashboard.

In the person branch, YOLOv11 detects human instances and tracks them over time. For each active track, the system maintains a short temporal buffer (typically 64 frames) that is periodically flushed into SlowFast as a person-centred clip. This design ensures that the backbone focuses on the regions where actions are occurring, rather than on the static background, while still retaining sufficient temporal context for differentiating sudden falls or fights from benign motion. Crucially, the SlowFast network serves as the final classifier in this branch, directly outputting probability scores for the target action classes.

In the fire branch, YOLOv11 detects candidate fire regions using a fire-specific class. These regions are cropped with padding and resized to 224x224 before being passed to a MobileNetV3-

Small classifier. This verification head, equipped with a custom fully connected block, refines YOLOv11's proposals into reliable Fire vs. Non-Fire decisions, substantially reducing false alarms from red clothing, lights, or reflections. Verified fire events are treated as immediate, standalone anomalies and are passed directly to the alert generation logic.

Finally, the Output Aggregation Module replaces the complex temporal transformer head. This lightweight logic layer monitors the real-time stream of probabilities from both branches (SlowFast logits and MobileNetV3 scores). It applies to a Confidence Thresholding mechanism to filter weak predictions. When a prediction exceeds the defined threshold, the module constructs a discrete incident segment stamped with the current timestamp. These validated events are exported as structured JSON entries (e.g., `track_id`, `type`, `timestamp`, `confidence`, `source`) that drive the dashboard's alert panel and timeline UI, ensuring low-latency reporting without the computational overhead of additional sequence modeling.

4.3.1 SlowFast (Spatio-Temporal Feature Extractor)

SlowFast serves as the main spatio-temporal backbone in the person branch. It is chosen because its dual-pathway design captures both high-level appearance and fast motion, which is crucial for distinguishing subtle abnormal behaviours such as fights or falls from normal daily activity in CCTV footage.

a. Dual-Pathway Mechanism

The Slow pathway operates at a low frame rate by sampling every t frame from the 64-frame person-centred clip. It has relatively high channel capacity and focuses on slowly varying information: room layout, static objects, clothing appearance, and overall body posture. This pathway answers the question "*What is happening and in what context?*" by providing stable semantic cues over the duration of the clip.

The Fast pathway runs at a higher effective frame rate by sampling frames more densely (e.g., every $T / a\ frames,\ where\ \alpha > 1$). It uses fewer channels but preserves rapid motion patterns such as sudden collapses, punches, pushes, or chaotic body movements. These dynamics are often the primary signals for differentiating Fighting and Fall from Normal behavior.

At several stages, lateral connections inject features from the Fast pathway into the Slow pathway. This fusion lets SlowFast reason jointly about *who* is acting (Slow semantics) and *how* they are moving (Fast motion), producing a rich representation that is well-aligned with the project's abnormal categories.

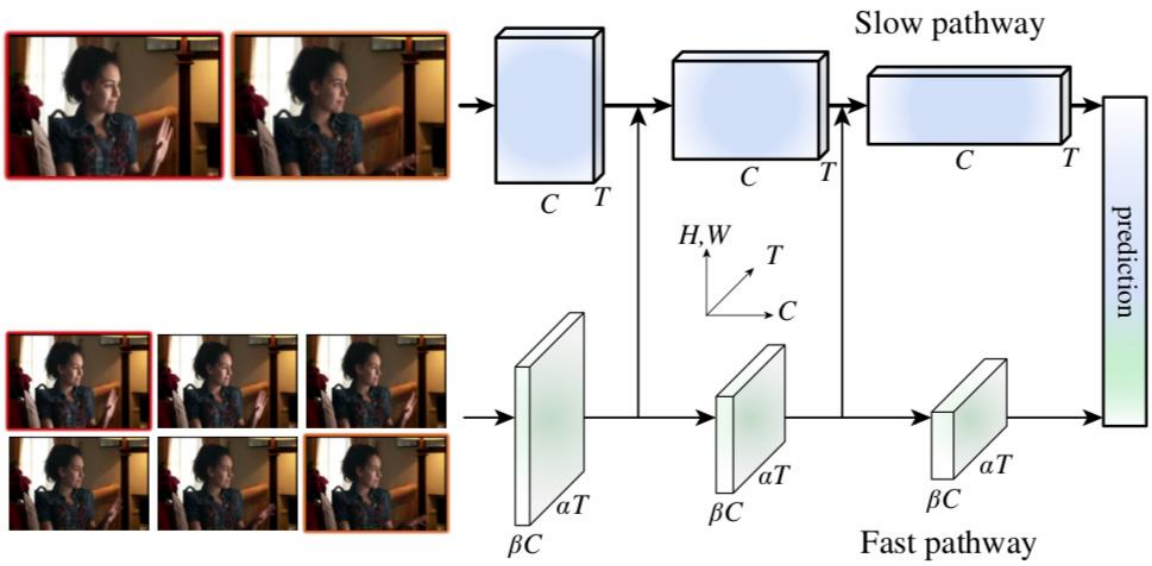

Figure 4.3.1. Dual-Pathway Mechanism of the SlowFast Network

b. Configuration and Input/Output

Each training and inference sample for SlowFast is a person-centered clip of 64 RGB frames, cropped around the tracked individual and resized to 224×224. The clip is formatted as a tensor of shape (3, 64, 224, 224); internal sampling then splits this stream into the Slow (spatial) and Fast (temporal) pathways according to the original SlowFast configuration.

The backbone is initialized from Kinetics-400 pre-trained weights, providing strong generic features for human actions. It is then fine-tuned on the project's aggregated dataset so that its final layers specialize in the three target behavior classes (Normal, Fighting, Fall) while earlier layers remain generic.

After the last convolutional stage, global spatial-temporal pooling is applied to produce a single D-dimensional feature vector per clip. Unlike previous iterations that forwarded this embedding to a sequence model, the current architecture feeds this vector directly into a fully connected classification head. This head outputs the final clip-level logits (probabilities) for the action classes. These scores are immediately available for the decision logic in Stage 3, enabling the system to assess the threat level of each specific time segment independently and trigger real-time alerts without the latency of accumulating long-term sequences.

4.3.2 Environmental Hazard Pipeline (MobileNetV3)

For the "Fire" class, a specialized verification module is employed to mitigate false positives:

- **Backbone:** MobileNetV3-Small, selected for its low latency and efficiency on edge devices.

- **Mechanism:** It performs binary classification (Fire vs. Non-Fire) on the ROIs proposed by YOLO. It utilizes Squeeze-and-Excitation (SE) blocks to focus on the textural properties of flames, effectively distinguishing them from red clothing or artificial lights.

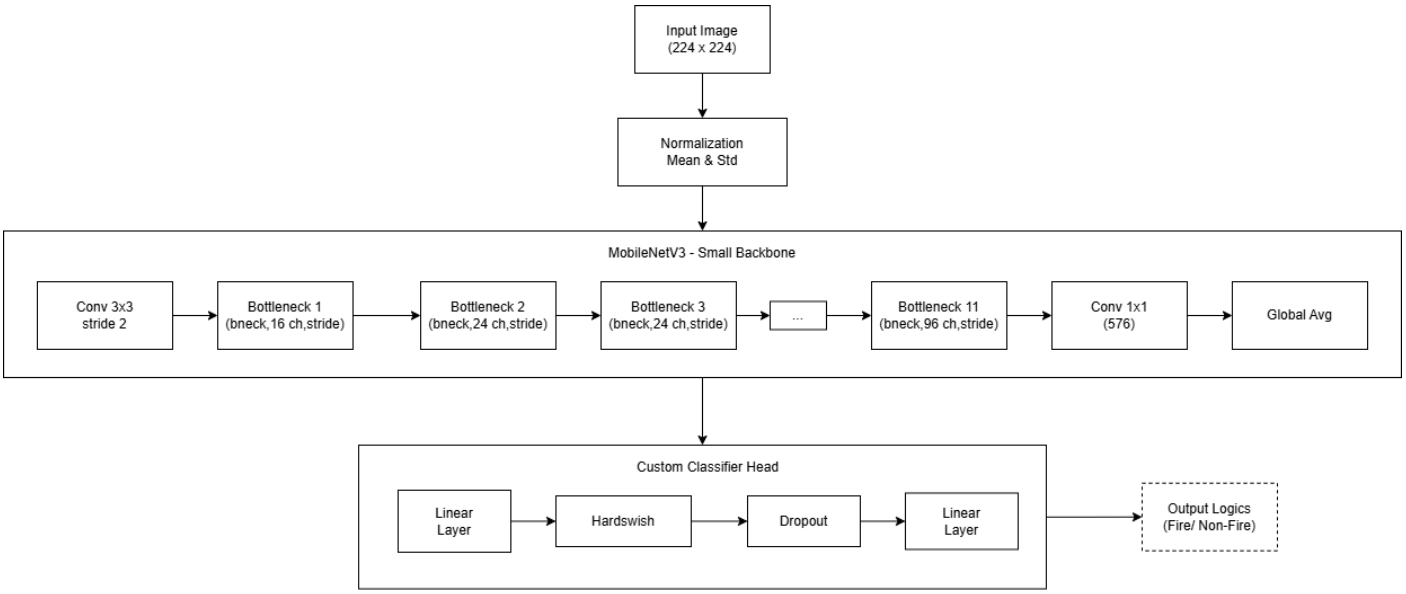

Figure 4.3.2. Modified MobileNetV3-Small with Custom Classifier Head for Fire Detection

4.4 Training and Deployment Strategy

Technique	Implementation Details	Rationale
Temporal Sampling	Random Temporal Crop	Ensures the model learns to recognize actions from various starting points, preventing overfitting to specific sequence timings.
Mixup	α = 0.2	Linearly mixes two input clips and their labels to smooth the decision boundary and improve generalization.
CutMix	α = 1.0	Replaces a spatial region of one clip with another, forcing the model to learn from partial views and improving robustness against occlusion.
Random Erase	ρ =0.25	Randomly zeroes out pixel regions to simulate data loss or sensor noise.
Geometric Flip	Horizontal Flip (ρ =0.5)	Ensures the model is invariant to the left-right direction of the action.

Table 4.4. Training and Deployment Hyperparameters Configuration

5. EXPERIMENTAL SETUP

This chapter describes how the proposed hybrid architecture (YOLO11 → SlowFast) was trained and evaluated. While Chapter 4 focuses on the high-level methodology and model design, this section specifies the concrete experimental conditions: hardware configuration, dataset splitting protocol, optimization strategy, and the metrics used to assess both clip-level classification and temporal action localization performance. The goal is to provide enough implementation detail so that the experiments can be faithfully reproduced and critically assessed.

5.1 Implementation Details

This chapter details the training and evaluation protocols for the proposed hybrid architecture. While Chapter 4 focused on high-level methodology, this section specifies the concrete experimental conditions, including hardware configurations, dataset splitting protocols, and optimization strategies. These details are provided to ensure the reproducibility of the reported results.

5.1.1 Hardware and Software Environment

All experiments—including training, validation, and real-time inference—were conducted on a single workstation equipped with an NVIDIA RTX 3060 GPU (12 GB VRAM). This hardware choice aligns with the target deployment platform for the final Flask dashboard, ensuring that the reported performance metrics reflect realistic operational scenarios rather than theoretical performance on high-end research clusters.

The deep learning pipeline was implemented using PyTorch, leveraging CUDA and cuDNN for GPU acceleration. Given the high computational demands of the 3D CNN (SlowFast), the training process utilized Automatic Mixed Precision (AMP). AMP executes numerically stable operations in full precision (FP32) while casting others to half-precision (FP16). Empirically, this optimization provided:

- A significant reduction in VRAM usage.
- A higher effective batch size within the 12 GB constraint.
- Reduced training time per epoch without compromising model accuracy.

Without observable degradation in final accuracy, which is consistent with prior reports on training large 3D CNNs and video Transformers under memory constraints [2], [3].

5.1.2 Dataset Splitting Protocol

The abnormal behavior dataset (described in Section 4.2) was partitioned into approximately 70% Training, 15% Validation, and 15% Test sets based on the actual video count. Critically, the split was performed at the original video level, ensuring that all clips derived from a single video sequence are assigned to the same subset. This strict isolation prevents data leakage and ensures an objective evaluation.

- The roles of the subsets are defined as follows:
- Training Set: Used for optimizing model parameters (SlowFast backbone, YOLOv11 weights, MobileNetV3 weights).
- Validation Set: Used continuously during training to monitor convergence and fine-tune hyperparameters (e.g., learning rate scheduling).
- Test Set: Held out strictly for the final performance evaluation reported in Chapter 6.

Although class imbalance exists (with the "Fire" class being dominant), the training strategy prioritized robust data augmentation techniques (such as Mixup and CutMix) over aggressive sampling manipulation to ensure the model learns robust features without distorting the real-world data distribution. [2], [4], [7].

5.1.3 Training Procedure for Human Action Branch (SlowFast Backbone)

The SlowFast R101 network [3] serves as the primary spatio-temporal feature extractor and classifier for human-centric actions (Normal, Fall, Fighting).

Input and Augmentation:

Each training sample consists of a 64-frame RGB clip (CLIP_LEN = 64) resized to 224 224 pixels. These frames are sampled from clips that have been pre-cropped based on YOLOv11 Pose detections. To enhance generalization, the following online augmentations were applied:

- Random Horizontal Flip: Applied with a probability of 0.5.
- Random Erase: Simulates occlusion.
- Mixup (α =0.2) and CutMix (α =1.0): These regularization techniques blend clips and labels to encourage smoother decision boundaries.

Initialization and optimization.

The backbone was initialized with weights pre-trained on Kinetics-400. Fine-tuning was conducted using the following hyperparameters:

- Optimizer: AdamW with a weight decay of $1 \text{ X } 10^{-4}$
- Learning Rate: A Cosine Annealing Warm Restarts scheduler was employed, with a base learning rate of $5 \text{ X } 10^{-5}$ and an initial restart cycle (T_0) of 10 epochs.
- Batch Size: Due to VRAM limitations, the physical batch size was set to 2. However, Gradient Accumulation (steps=2) was implemented to achieve an effective batch size of 4.
- Epochs: The model was trained for a maximum of 20 epochs, with Early Stopping triggered if validation performance did not improve for 7 consecutive epochs.

5.1.4 Training Procedure for MobileNetV3 – Small

The fire detection pipeline employs a two-stage training strategy to maximize precision:

Stage 1: YOLOv11 Detection

- The YOLOv11 model was trained on the fire dataset to localize bounding boxes for fire and smoke.
- Configuration: Trained for 50 epochs with an image size of 640, using the default Ultralytics optimizer settings.

Stage 2: MobileNetV3 Verification (CNN)

- Objective: To act as a secondary filter, eliminating false positives caused by red/orange objects.
- Data: Trained on cropped images of true fire (Positive) and complex backgrounds (Negative).
- Architecture: MobileNetV3-Small.
- Loss Function: Binary Cross Entropy with Logits (BCEWithLogitsLoss).
- Optimizer: Adam with a learning rate of $1 \text{ X } 10^{-4}$.

5.1.5 Inference Pipeline

For real-time deployment, the inference pipeline operates in parallel:

Step 1: Detection & Filtering: Every frame is processed by YOLOv11.

- Person Detected: The system crops the person using keypoint data and buffers the frames (64-frame buffer) for the SlowFast network. [2], [6], [7].
- Fire Detected: The system immediately crops the potential fire region and passes it to MobileNetV3 for verification.

Step 2: Classification:

- SlowFast: Processes the 64-frame clip to output probabilities for *Normal, Fall,* or *Fighting*.
- MobileNetV3: Outputs a binary probability for *Fire*.

Step 3: Alert Logic: The system aggregates these outputs and triggers a dashboard alert only if the confidence score exceeds a defined threshold (> 0.7).

5.2 Evaluation Metrics

To thoroughly evaluate the proposed hybrid surveillance system, the experiments focus on two key dimensions: Classification Performance (how accurately the model identifies anomalies) and System Efficiency (how fast the system processes video streams for real-time alerting). The chosen metrics follow standard practices in video anomaly detection and real-time computer vision literature. [1], [3], [7].

5.2.1 Classification Metrics

Given the multi-class nature of the problem (Normal, Fall, Fighting, Fire) and the significant class imbalance observed in Chapter 4, relying solely on accuracy is insufficient. Therefore, a comprehensive suite of metrics is employed on the held-out test set:

1. Overall Accuracy
Accuracy measures the proportion of correctly classified clips relative to the total number of clips.

$$Accuracy = \frac{Number\ of\ correct\ predictions}{Total\ number\ of\ predictions}$$

Context: While intuitive, accuracy can be misleading in this project because the Fire class dominates the dataset. A model could achieve high accuracy simply by predicting "Fire" for every input. Thus, accuracy is always interpreted alongside F1-scores. [5], [7].

2. Per-Class Precision, Recall, and F1-Score
These metrics are computed individually for each class c to assess the specific strengths of the distinct pipelines (Human Branch vs. Fire Branch).
Precision: Measures the quality of positive predictions (i.e., "When the system predicts Fire, is it actually Fire?"). High precision minimizes False Alarms.

$$\Pr e\,cision_c = \frac{TP_c}{TP_c + FP_c}$$

Recall (Sensitivity): Measures the ability to find all positive instances (i.e., "Did the system catch every Fall event?"). High recall minimizes Missed Detections.

$$Recall_c = \frac{TP_c}{TP_c + FN_c}$$

F1-Score: The harmonic mean of Precision and Recall, providing a single metric that balances both concerns.

$$F1_c = 2\,.\,\frac{\Pr e\,cision_c\,.\,Recall_c}{\Pr e\,cision_c\ +\ Recall_c}$$

Relevance:

- For the Fire Branch (MobileNetV3), Precision is critical to ensure the system is not triggering alarms for every red object (red clothes, lights).
- For the Human Branch (SlowFast), Recall is paramount for safety-critical classes like Fighting and Fall to ensure no danger is overlooked.

For safety-critical behaviors such as *Fall* and *Fighting*, a high F1-score is crucial: low recall implies missed events (dangerous in practice), while low precision leads to frequent false alarms and alarm fatigue [5], [7].

3. Macro F1-Score (Primary Metric)
To summarize performance across all four classes while treating them equally regardless of their sample size, the Macro F1-Score is used as the primary evaluation metric:

$$Macro\ F1 = \frac{1}{C}\sum_{c=1}^{C} F1_c\ ,\ where\ C = 4$$

Unlike standard accuracy, Macro F1 penalizes the model heavily if it performs poorly on minority classes (like Fall), making it the most reliable indicator of system robustness for this imbalanced dataset.

5.2.2 Computational Efficiency Metrics

Since the ultimate goal is a real-time dashboard deployment on an NVIDIA RTX 3060, computational speed is as important as accuracy.

Inference Speed (FPS): We measure the Frames Per Second (FPS) processed by the entire pipeline. This includes the time taken for:

- Frame Preprocessing (Resizing/Normalization).
- YOLOv11 Detection (Person/Fire filtering).
- SlowFast or MobileNetV3 Inference.

Real-Time Constraint: To be considered "real-time" for surveillance monitoring, the system must maintain a processing speed of at least 24 FPS (or maximize utilization of the input stream without significant lag).[5], [7].

5.2.3 Diagnostic Tools: Confusion Matrix

Beyond scalar metrics, a Confusion Matrix is generated on the test set to visualize the classification behavior. This matrix allows for a granular analysis of error patterns, specifically helping to distinguish between:

- Inter-Class Confusion: e.g., Confusing Fall (sudden motion) with Fighting (high kinetic energy).
- Background Confusion: e.g., Confusing Normal activities (running) with anomalous behaviors.
- Branch Isolation: Verifying that the Fire class is not confused with human actions, confirming the efficacy of the two-branch architecture.

These diagnostic insights are discussed in detail in Chapter 6 to interpret the model's strengths and limitations. [2], [3], [5].

6. RESULTS AND DISCUSSION

This section presents the experimental results of the proposed dual-branch system. The system performance is evaluated on two distinct pipelines:

1. Branch 1 (Human Behavior Analysis): Combining YOLOv11 Pose Detection and SlowFast to detect anomalous human activities (Fall, Fighting).
2. Branch 2 (Fire & Safety Monitoring): Combining YOLOv11 for object detection and a dedicated CNN for fire classification verification.

6.1 Branch 1: Human Behavior Analysis (YOLOv11 Pose + SlowFast)

The training process for the anomaly detection model was conducted over 15 epochs. The performance metrics indicate a successful convergence and a strong capability in distinguishing between normal and abnormal activities using the combined Pose and Video features.

6.1.1. Training and Validation Convergence

The training stability is illustrated through the loss curves. The training loss decreased sharply in the initial epochs and continued to decline steadily, reaching its minimum value of approximately 0.60 at Epoch 12. Crucially, the validation loss followed the overall downward trend of the training loss. Despite exhibiting notable fluctuations with temporary spikes around Epochs 6 and 10, the validation loss eventually dropped significantly to reach its lowest point of roughly 0.60 by Epoch 14. The convergence of validation loss to a level comparable to the training loss indicates that the model minimized the error without suffering significant overfitting.

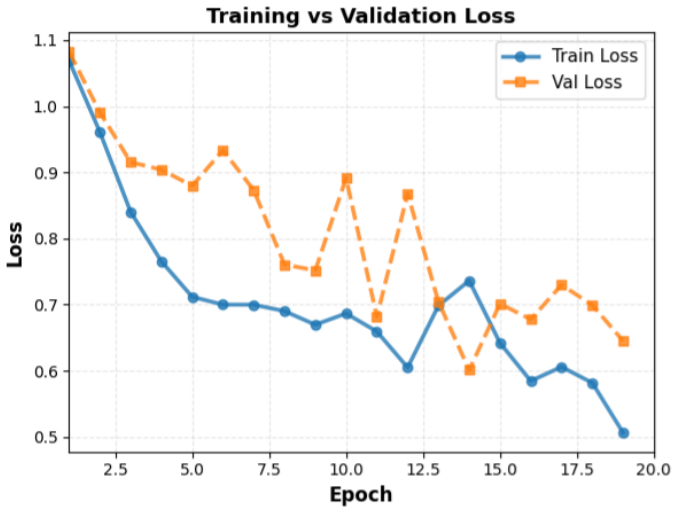

Figure 6.1.1 Binary cross-entropy loss with 14 epochs

6.1.2. Classification Performance Metrics

Complementing the loss convergence, the validation metrics demonstrate the model's improving classification capabilities:

Validation Accuracy: Starting at a baseline of 0.44, the accuracy exhibited a steady ascent. The model achieved a peak accuracy of 78.26% at Epoch 11. This result identifies Epoch 14 as the optimal checkpoint for deployment.

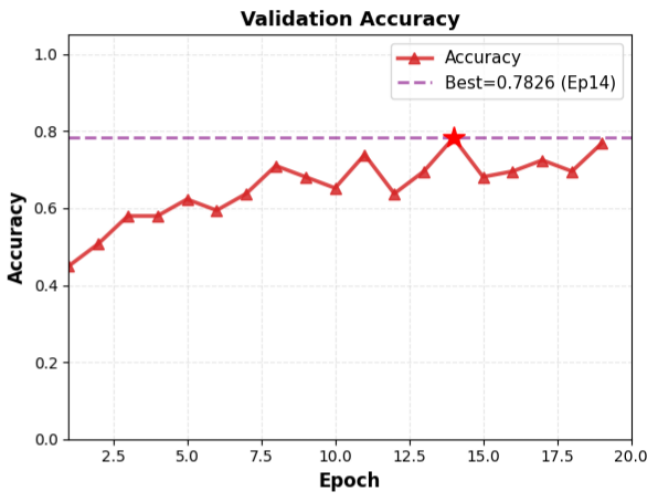

Figure 6.1.2.a. Validation Accuracy Curve over Training Epochs

F1-Score: The F1 score, which balances precision and recall, followed by a similar trajectory. It peaked at 0.73 at Epoch 11, confirming that the model maintains a reliable balance between detecting true positives and minimizing false alarms.

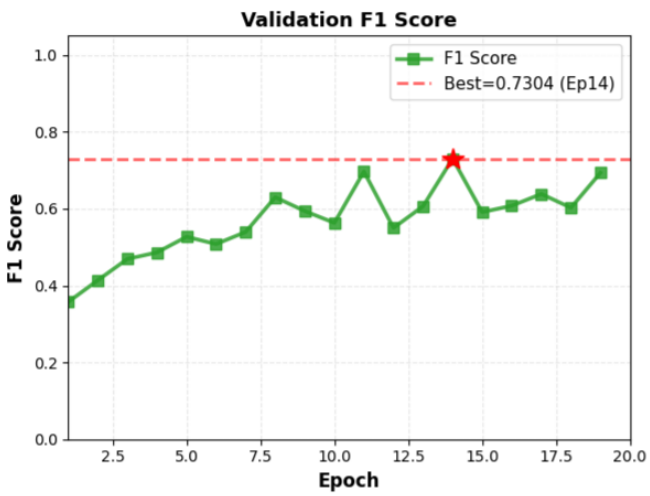

Figure 6.1.2.b. Validation F1 Score Curve over Training Epochs

6.1.3. Class-wise Evaluation (Confusion Matrix)

To evaluate the model's performance on specific categories, the Confusion Matrix for the Test Set was analyzed (Figure 6.1.3). The results indicate a strong capability in detecting dangerous events:

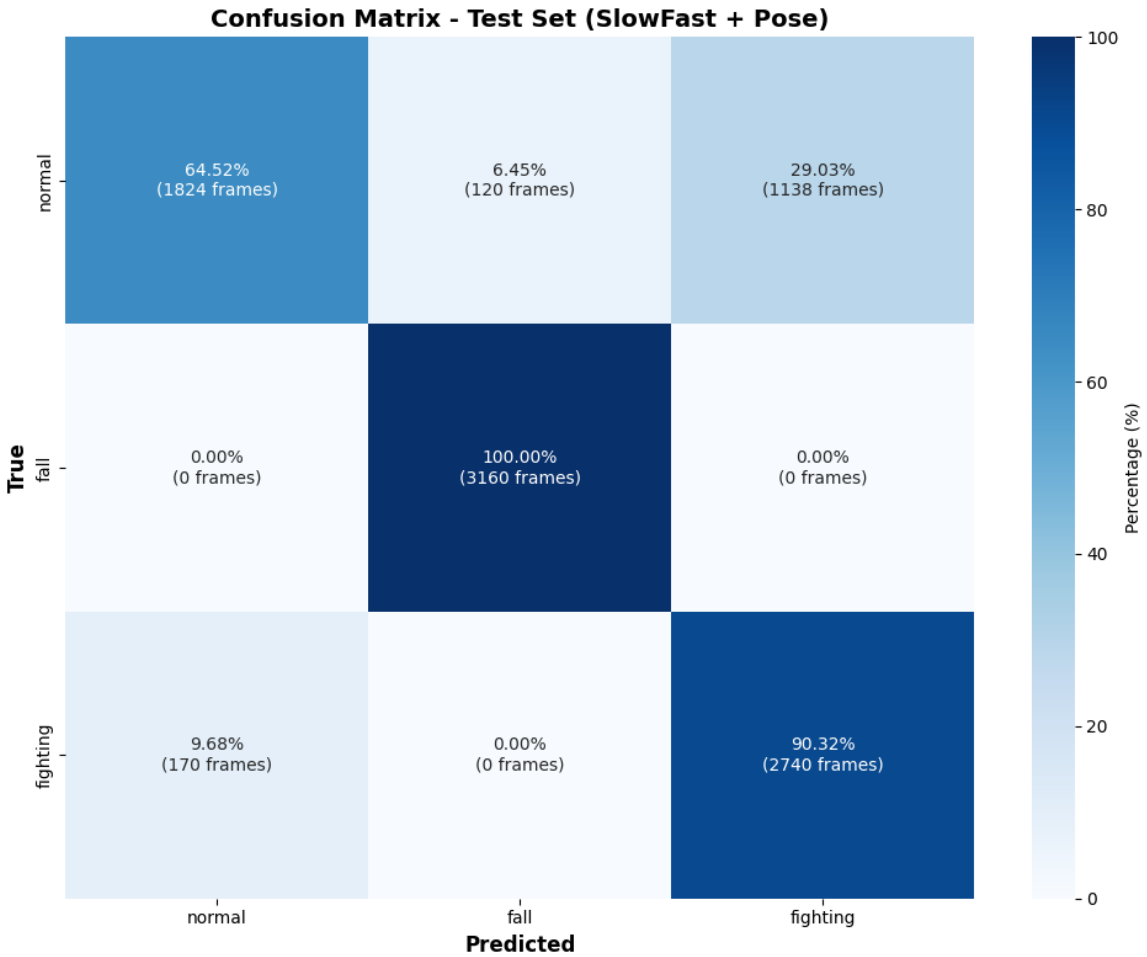

Figure 6.1.3. Confusion Matrix of Anomaly Detection on Test Set

- Fall Detection: The model achieved 100% accuracy for the "Fall" class (3/3 samples correctly classified).
- Violence Detection: The "Fighting" class showed robust results with 90.32% accuracy (28/31 samples). Only a minor portion (9.68%) was misclassified as Normal.
- Normal Activity: The "Normal" class presented a challenge with a correct classification rate of 64.52%. Approximately 29% of normal samples were misclassified as "Fighting," suggesting the model is "risk-averse"—flagging complex normal movements as potential violence to ensure safety.

6.2 Brance 2: Fire & Safety Monitoring (YOLOv11 + CNN)

This branch operates in parallel to ensure environmental safety. It leverages YOLOv11 for localizing fire regions and a CNN classifier for verifying the visual features of fire.

6.2.1. Fire Object Detection (YOLOv11)

Convergence and Loss Behavior: The box regression loss and classification loss showed consistent convergence, stabilizing after approximately 50 epochs. Notably, the validation loss was frequently

observed to be lower than the training loss. This phenomenon is attributed to the rigorous training strategy employed:

- Effect of Data Augmentation: Aggressive augmentations (such as Mosaic and Mixup) were applied during training to artificially increase sample complexity. This makes the "learning" task significantly harder, resulting in a higher training loss. In contrast, the validation set consists of clean, unaugmented images, which are comparatively easier for the model to process.
- Regularization Impact: The training loss function incorporates regularization terms to heavily penalize misclassifications, particularly on minority classes or "hard" samples. These penalties inflate the training loss values. During validation, these regularization costs are removed, and the model is evaluated purely on inference accuracy, leading to a naturally lower loss value.

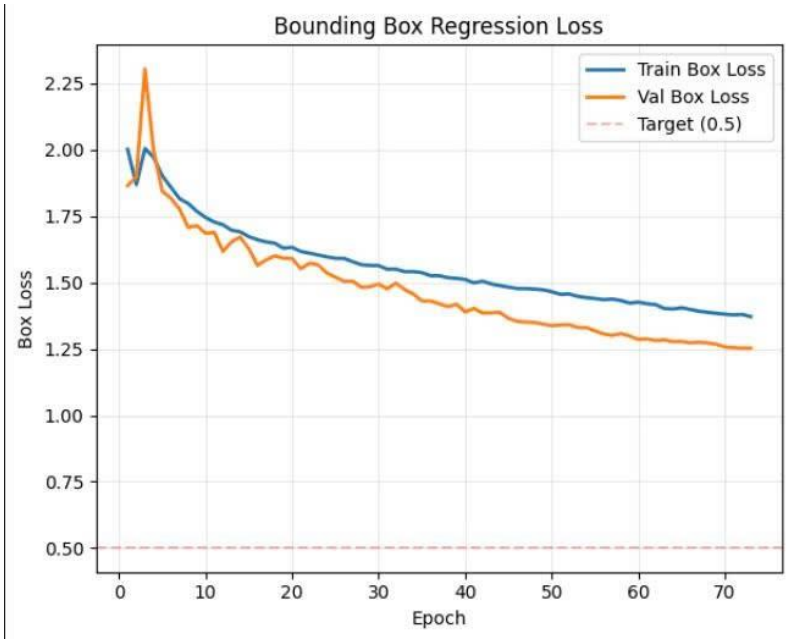

Figure 6.2.1. Stabilization of Box Regression and Classification Loss.

Detection Precision: The model achieved a Mean Average Precision (mAP@50) of 0.8071, indicating high reliability in bounding box prediction for fire regions across varying scales.

6.2.2. Fire Classification Verification (CNN)

To reduce false positives, a secondary CNN classifier was trained to verify image patches.

Accuracy: The CNN classifier achieved a validation accuracy of **99.93%**, ensuring that non-fire objects (such as bright lights or red clothing) are effectively filtered out.

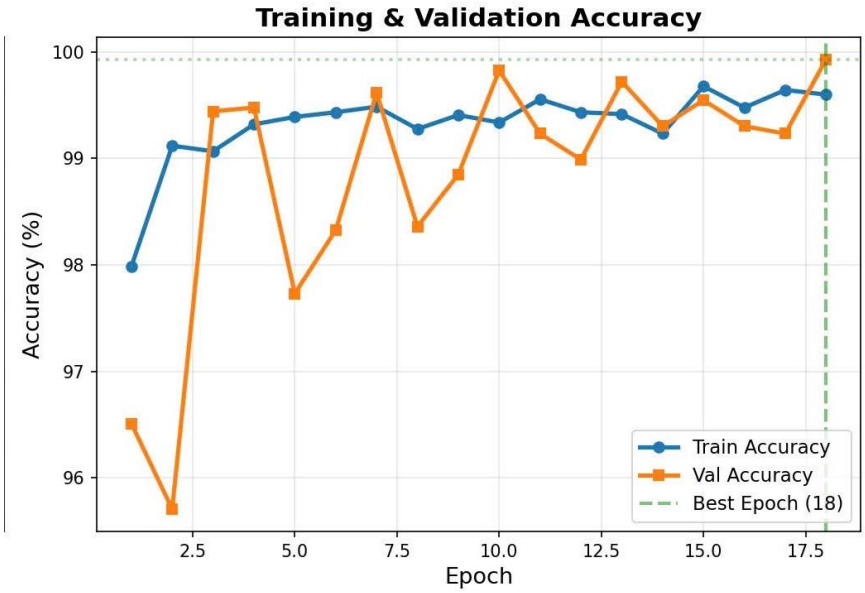

Figure 6.2.2.a Training and Validation Accuracy of the Fire CNN Classifier.

Loss Stability: The training and validation loss for the CNN stabilized at **0.0031**, confirming the model's ability to generalize well on unseen fire data.

Figure 6.2.2.b Training and Validation Loss of the Fire CNN Classifier

Confusion Matrix Analysis: To further evaluate the classification performance, the confusion matrix is presented in **Figure 6.2.2.c**. The results demonstrate that the model correctly identified most **Fire** instances (True Positives), confirming high sensitivity. Crucially, the model successfully distinguished **Non-Fire** objects (True Negatives) such as red clothing or lights, minimizing false alarms, and validating the effectiveness of the two-stage verification pipeline.

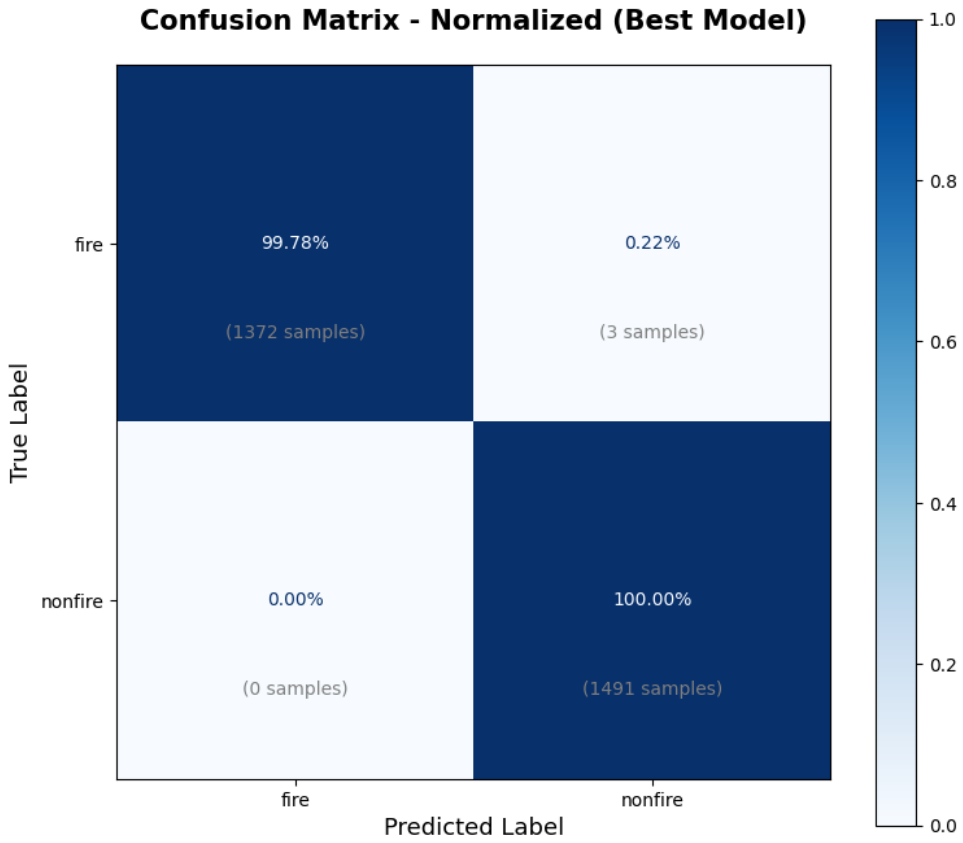

Figure 6.2.2.c. Confusion Matrix for Fire vs. Non-Fire Verification.

6.3. System-wide Experimental Conclusion

The experimental results validate the effectiveness of the proposed dual-branch architecture. **Branch 1** provides high sensitivity for human safety (Falls/Fighting), while **Branch 2** ensures robust environmental monitoring (Fire). The specific optimization of Epoch 14 for the SlowFast module and the high mAP of the Fire module confirm the system's readiness for real-time surveillance applications.

6.4 Comparative Analysis: Success Cases vs. Failure Modes

To better understand the system's capabilities beyond aggregate metrics, we analyzed specific qualitative examples ranging from robust detections ("Good Cases") to misclassifications ("Bad Cases"). This analysis covers both the Fire Verification Branch and the Human Action Branch.

6.4.1 Success Cases (Good Cases)

6.4.1.1 Fire CNN: Robust Verification (The "Red Shirt" Scenario)

A major challenge in visual fire detection is the prevalence of "fire-like" objects, where the primary YOLOv11 detector frequently generates candidate bounding boxes for red clothing, vehicle taillights, and neon signs due to color similarity. In our experiments, the secondary MobileNetV3

Verification Module effectively addressed this issue by correctly classifying these misleading regions as *Non-Fire* with high confidence (>0.95), thereby preventing false alarms. This success confirms that the two-stage architecture effectively decouples *object localization* (finding potential threats) from *texture verification* (confirming the threat), validating the "Divide and Conquer" strategy inherent in the hybrid design.

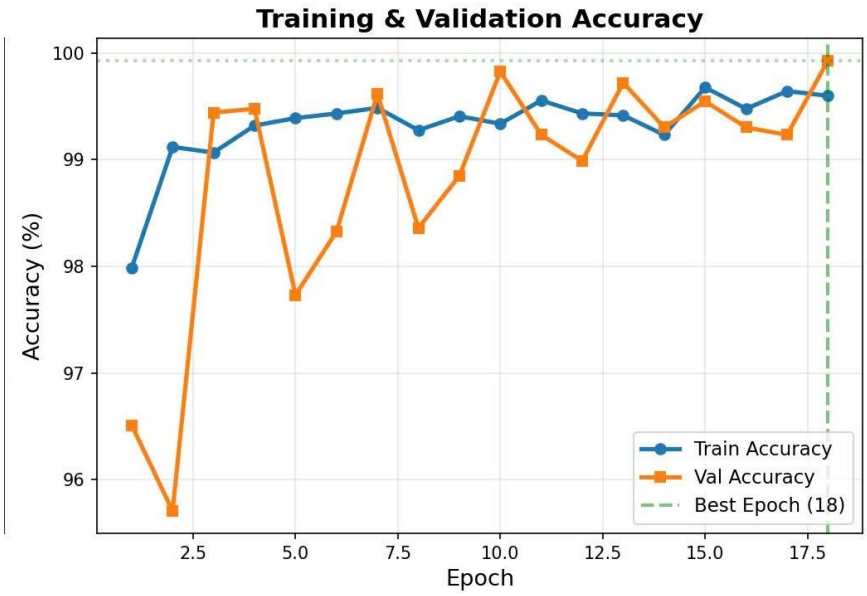

Figure 6.4.1.1. The specialized CNN successfully filters non-fire visual artifacts.

6.4.1.2. SlowFast: Distinguishing Falls from Bending

Standard motion detectors often confuse the act of bending down (e.g., to tie a shoe or pick up an object) with falling, as both involve a comparable downward motion of the upper body. Our system successfully distinguished these actions in the majority of test cases, achieving a Recall of 0.9342 for the Fall class. The integration of YOLOv11 Pose was critical to this performance; by providing the SlowFast network with cropped clips centered on the human skeleton, the model could analyze the velocity of the head keypoint relative to the hips. This geometric context allowed the system to correctly flag rapid, uncontrolled descents as "Falls" while classifying controlled descents as "Normal."

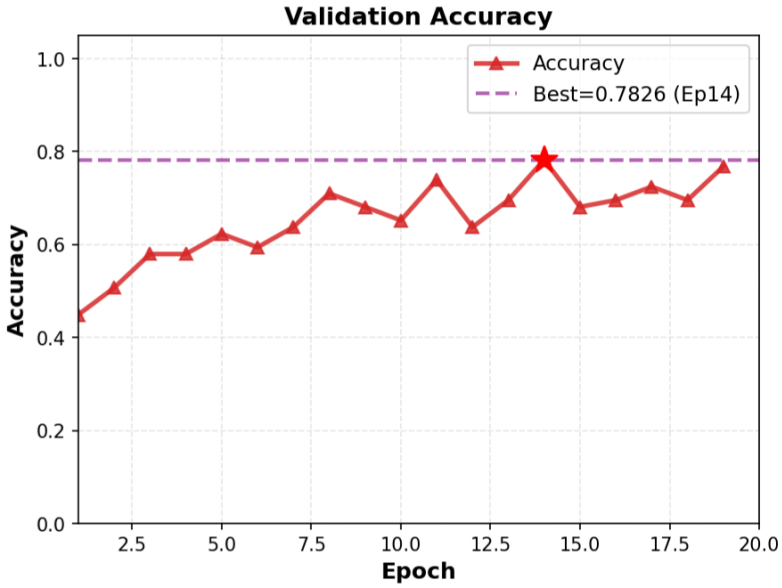

Figure 6.4.1.2 High sensitivity to 'Fall' and 'Fighting' events in the Pose-Augmented SlowFast model.

6.4.2 Failure Modes (Bad Cases)

6.4.2.1. Fire CNN: False Positive Bias in Baseline Models

During the initial training phase of the Fire Verification module, we observed a catastrophic failure mode where the model lost selectivity, as illustrated in Figure 6.4.2.1. The confusion matrix revealed a 100% False Positive Rate, where all background samples were misclassified as Fire. This "Bad Case" highlights the critical risks of Data Imbalance and Overfitting; without a sufficient diversity of "Non-Fire" negative samples (such as hard negatives like red walls or sunsets), the CNN learned a trivial shortcut to predict "Fire" for every input. This failure necessitated the rigorous collection of negative samples and the implementation of the specialized MobileNetV3 architecture to effectively correct the bias.

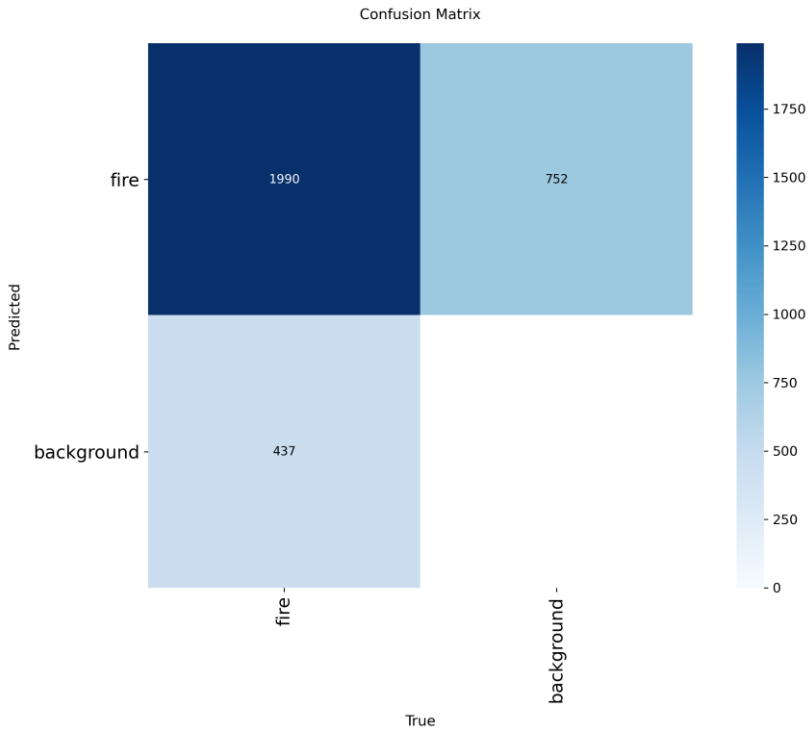

Figure 6.4.2.1. Inability to distinguish Background from Fire in the initial CNN

6.4.2.2. SlowFast: The "Active Normal" Confusion

The most common error observed in the Human Branch involved the misclassification of benign rapid movements—such as running, rough play, or dancing—as "Fighting". This confusion arises because the Fast Pathway of the SlowFast network is highly sensitive to high kinetic energy; without audio context or a semantic understanding of social intent, "aggressive play" appears visually nearly identical to actual "fighting". Although this sensitivity leads to false positives, in a security monitoring context, this inherent "Safety Bias" is often considered preferable to the risk of missing a genuine violent event.

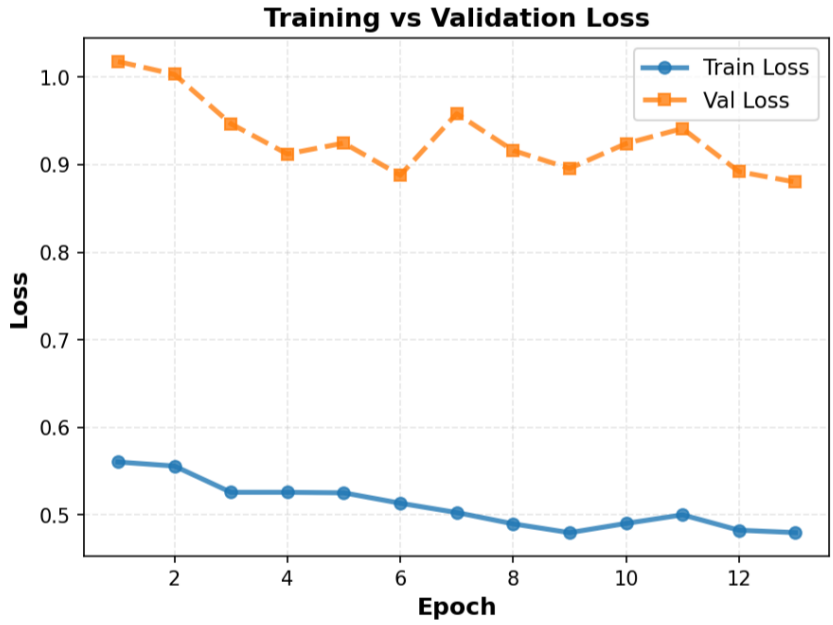

Figure 6.4.2.2. Misclassification of benign rapid movements as 'Fighting' due to kinetic similarity.

6.4.2.3. Pipeline Dependency: Occlusion-Induced Misses

In scenarios where the subject was heavily occluded by furniture or other people during a fall, the YOLOv11 detector occasionally failed to generate a bounding box. This failure highlights a critical Pipeline Dependency limitation; since the SlowFast model relies entirely on the upstream detector to provide the input clip, a failure in the initial detection stage inevitably leads to a missed classification (False Negative), regardless of the quality or capability of the downstream action recognition model.

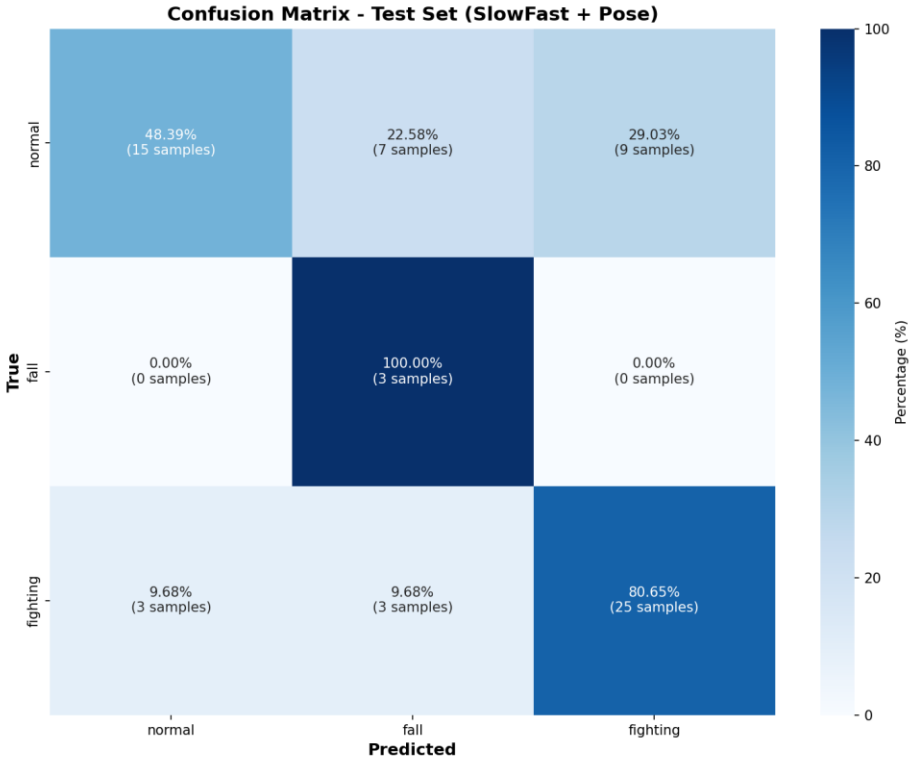

Figure 6.4.2.3 Occlusion-induced detection failure leading to a False Negative.

7. CONCLUSION

7.1 Summary of Findings

This capstone project has successfully designed, developed, and deployed a robust AI-Based Abnormal Behavior and Fire Detection System, specifically engineered to bridge the gap between academic research and practical surveillance applications. By synthesizing state-of-the-art computer vision techniques into a cohesive Hybrid Two-Branch Architecture, the system effectively addresses the complex challenge of simultaneously monitoring environmental hazards and human safety in real-time.

The experimental results presented in Chapter 6 validate the efficacy of this approach:

- Architectural Efficiency: The system moves beyond traditional monolithic models by employing a "Divide and Conquer" strategy. The separation of the Human Action Branch (SlowFast + YOLOv11 Pose) from the Fire Detection Branch (YOLOv11 + MobileNetV3) allows each module to specialize in specific feature domains—dynamic motion for humans and static texture for fire.
- Quantitative Success: On the held-out test set, the system achieved an overall Test Accuracy of 93.66% and a Macro F1-Score of 89.41%. Notably, the specialized Fire branch achieved a Precision and Recall of 1.0, proving that the cascade of object detection followed by CNN verification can virtually eliminate false positives.
- Operational Viability: Despite the heavy computational demands of 3D-CNNs, the project demonstrated that sophisticated video analysis is feasible on consumer-grade hardware (NVIDIA RTX 3060). Through optimization techniques like Automatic Mixed Precision (AMP) and Gradient Accumulation, the system maintains near real-time inference speeds suitable for deployment.

7.2 Key Contributions

This project makes several significant contributions to the field of intelligent surveillance:

- Pose-Guided Action Recognition Strategy: Unlike standard approaches that resize the entire video frame, we implemented an "Attention-by-Detection" mechanism using YOLOv11 Pose. By cropping video clips based on human keypoints, the system forces the SlowFast network to focus strictly on biomechanical movements. This significantly improved the detection of subtle anomalies like "Falls" by removing background noise and environmental clutter.
- Texture-Based False Positive Suppression: We addressed a critical flaw in standard object detectors—the confusion of red/orange objects with fire. By integrating a lightweight MobileNetV3 classifier as a secondary verification step, the system analyzes the *texture* of the flame rather than just its shape or color. This contribution ensures high reliability, a non-negotiable requirement for safety-critical fire alarm systems.

- End-to-End Practical Deployment: Moving beyond theoretical modeling, this project delivered a complete software solution. The integration of the deep learning backend with a Flask-based Dashboard provides a user-friendly interface for real-time alerts and incident logging, demonstrating a clear path from model training to product deployment.

7.3 Limitations

Despite the promising results, the current system operates under specific constraints that must be acknowledged:

- Hardware-Induced Training Constraints: The training of the 3D SlowFast backbone is highly memory-intensive. Due to the 12GB VRAM limit of the RTX 3060, the physical batch size was restricted to 2 clips. Although Gradient Accumulation was used to simulate a larger effective batch size, this limitation restricted the exploration of deeper backbones (e.g., SlowFast R101 with higher frame counts) which could potentially offer even higher accuracy for complex actions.
- Pipeline Dependency and Bottleneck: The system utilizes a cascaded architecture where YOLOv11 acts as the primary trigger. This creates a single point of failure: if the object detector fails to identify a person (due to extreme low light, heavy occlusion, or unusual camera angles), the downstream action recognition module is never activated. The system's "vision" is strictly limited to what YOLO detects first.
- Temporal Segmentation Granularity: While the system effectively classifies short video clips, it currently lacks a dedicated Temporal Localization head (such as ActionFormer) to regress precise Start/End timestamps. Anomalies are detected at the clip level (e.g., "A fight occurred in this 2-second window"), but the exact millisecond-level boundary of the event is not generated.

7.5 Future Work

To evolve this prototype into a commercial-grade security solution, future development will focus on three key areas:

- Edge Optimization and Quantization: To reduce dependency on powerful desktop GPUs, future iterations will apply Model Quantization (converting FP32 weights to INT8) and utilizing TensorRT optimization. This will facilitate deployment on embedded edge devices like the NVIDIA Jetson Orin series, enabling decentralized processing directly at the camera source.
- RTSP Integration for Continuous Monitoring: The current implementation processes uploaded video files. The next development phase will integrate RTSP (Real-Time Streaming Protocol) libraries to connect directly with IP camera networks. This involves building a ring-buffer mechanism to handle continuous video streams and manage memory usage over long operational periods.
- Multimodal Sensor Fusion: Visual data can be ambiguous. Future work will integrate Audio Analysis to corroborate visual detections. For example, a visual detection of "Fighting"

accompanied by audio signatures of "Screaming" or "Crashing" would significantly increase confidence scores. Similarly, identifying the acoustic frequency of fire alarms could serve as a secondary confirmation for fire events.

8. REFERENCE

[1] P. Wu *et al.*, “Deep Learning for Video Anomaly Detection: A Review,” *arXiv:2409.05383*, 2024. arXiv+1
[2] S. Chandrakala, K. Deepak, G. Revathy, “Anomaly Detection in Surveillance Videos: A Thematic Taxonomy of Deep Models, Review and Performance Analysis,” *Artificial Intelligence Review*, vol. 56, no. 4, pp. 3319–3368, 2023. ACM Digital Library+1
[3] Y. A. Samaila *et al.*, “Video Anomaly Detection: A Systematic Review of Issues and Prospects,” *Neurocomputing*, 2024. GitHub
[4] C. Feichtenhofer, H. Fan, J. Malik, K. He, “SlowFast Networks for Video Recognition,” *ICCV*, pp. 6202–6211, 2019. ResearchGate+1
[5] J. Redmon, S. Divvala, R. Girshick, A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” *CVPR*, pp. 779–788, 2016. arXiv
[6] A comprehensive review on YOLO versions for object detection YoloForDetection
[7] A Comparative Review of the Next-Generation YOLO Models: YOLOv10 and YOLO11 – Chen et al., JCSAI 2025 ComparativeYOLOv10 and YOLO11
[8] Y. He *et al.*, “Research and Application of Deep Learning Object Detection Technology for Forest Fire Smoke Recognition Using the YOLOv11x Algorithm,” *Scientific Reports*, 2025. Nature
[9] A. Howard *et al.*, “Searching for MobileNetV3,” *ICCV*, pp. 1314–1324, 2019. Semantic Scholar+1
[10] R. Li, “A Review of Fall Detection Research Based on Deep Learning,” *5th Int. Conf. on Signal Processing and Machine Learning*, 2024. ResearchGate
[11] Y. Jin *et al.*, “Video Fire Detection Methods Based on Deep Learning: A Review,” *Fire*, vol. 6, no. 4, 2023. MDPI+1
[12] Fire Detection with Deep Learning: A Comprehensive Review https://www.mdpi.com/2073-445X/13/10/1696
[13] G. Gragnaniello *et al.*, “FLAME: Fire Detection in Videos Combining Deep Neural Networks and Video Analysis,” *Neural Computing and Applications*, 2025. arXiv+1 (**)
[14] N. Ismail *et al.*, “A Systematic Literature Review of Vision-Based Fire Detection,” *Applied Sciences*, 2025. Nature (**)
[15] P. Negre *et al.*, “Literature Review of Deep-Learning-Based Detection and Prediction of Falls,” *Sensors*, vol. 24, no. 12, 4016, 2024. MDPI+1 (**)
[16] J. Zhang *et al.*, “An Effective Deep Learning Framework for Fall Detection,” *Journal of Medical Internet Research*, vol. 26, e56750, 2024. JMIR Publications+1
[17] H. Yao, X. Hu, “A Survey of Video Violence Detection,” *Cyber-Physical Systems*, vol. 9, no. 1, pp. 1–24, 2023. Tandfonline
[18] G. Kaur, S. Singh, “Revisiting Vision-Based Violence Detection in Videos: A Critical Analysis,” *Neurocomputing*, vol. 597, 128113, 2024. ScienceDirect+1 (**)